*Journal of Statistical Software*

*MMMMMM YYYY, Volume VV, Issue II.* *doi: 10.18637/jss.v000.i00*

# factorlasso: Hierarchical Clustering Group LASSO (HCGL) with Cluster-Pooled Sign Derivation for Multi-Asset Factor Models in Python

**Artur Sepp**
LGT Bank

**Mika A. Kastenholz**
LGT Bank

**Abstract**

Multi-asset factor models must be estimated from short samples at low signal-to-noise, where the interpretability of the loadings depends on cluster structure, on economically sensible loading signs, and on loading magnitudes. To our knowledge, no surveyed sparse-regression package combines internal cluster discovery, cell-level sign constraints, and prior-centred shrinkage. For portfolio optimisation and risk decomposition, practitioners must assemble these components by hand and can break the alpha-beta consistency that capital market assumptions (CMAs) require. The **factorlasso** Python package supplies the combination through a **scikit-learn**-style `fit`/`predict` interface built on **CVXPY**, in which clusters are discovered from the response correlation matrix, a closed-form residual sum-of-squares identity vectorises a noise-floor $t$ statistic gate, and the resulting sign matrix constrains a prior-centred sparse-group LASSO with adaptive penalty reweighting. On a data-generating process calibrated to a publicly reproducible 102-asset multi-asset exchange-traded-fund universe with known true loadings, three competing Python packages match **factorlasso** on $\beta$-MSE and out-of-sample fit. Yet only its sign- and prior-constrained configuration recovers the collinear credit factor that zero-centred shrinkage pushes into equity. The package ships a reproducible simulation and application harness and is available from the Python Package Index under the GNU General Public License.

*Keywords*: sparse regression, group LASSO, sign constraints, hierarchical clustering, adaptive LASSO, factor model, multi-response regression, Python, **CVXPY**.

## 1. Introduction

Factor models are central to quantitative finance, underpinning the commercial risk systems in industry use (among others, MSCI Barra, Axioma, and Bloomberg's multi-asset-class mod-

els). These systems decompose asset returns into a few common factors and an idiosyncratic residual (Rosenberg and McKibben 1973; Connor 1995). Practitioners who build factor-risk and capital-market-assumption systems must estimate the loadings of $N$ asset returns on $M$ tradable factors over a sample of length $T$, and they need software that produces loadings stable enough to feed a portfolio optimiser. In financial applications the estimation regime is adverse on four counts. First, the sample is short by statistical standards, because most core cross-asset indices and funds launched in the late 2000s and 2010s, giving 5 to 25 years of data. Second, the factors are often strongly correlated, so the design carries severe multicollinearity. Third, the cross-section is moderate ($N \in [50, 200]$ at the multi-asset-class level). Fourth, the true coefficient matrix $\boldsymbol{\beta} \in \mathbb{R}^{N \times M}$ exhibits a cluster structure that the practitioner does not know a priori, and traditional provider-based asset classifications are a weak proxy for the interdependence of assets. The cluster structure matters: assets in the same economic group (regional equity, investment-grade credit, hedge funds, and so on) typically share signs on the dominant factors, and an estimator that exploits this structure can outperform one that treats each asset independently.

Sparse regression has a long history of methodology and software for each component of this problem in isolation. The LASSO of Tibshirani (1996) produces a sparse coefficient vector via $\ell_1$ penalisation, the group LASSO of Yuan and Lin (2006) produces sparse group selection via the $\ell_2$-of-each-group penalty, the sparse group LASSO of Simon, Friedman, Hastie, and Tibshirani (2013) mixes the two, and the adaptive LASSO of Zou (2006) and its group extension by Wang and Leng (2008) reweight the penalty by univariate evidence to recover the oracle property. The univariate-guided LASSO of Chatterjee, Hastie, and Tibshirani (2025) uses univariate slope signs as hard constraints on the multivariate problem. The biobank-scale extension of Richland, Kiiskinen, Wang, Lu, Narasimhan, Hastie, Rivas, and Tibshirani (2025) combines univariate sign constraints with adaptive reweighting on genome-wide data with millions of candidate predictors. The high-dimensional sparse-recovery view of penalised regression goes back to Vattikuti, Lee, Chang, Hsu, and Chow (2014), who apply compressed sensing to genome-wide association data and document a sharp sample-size threshold for support recovery. Hierarchical clustering of the response correlation matrix, as in Bondell and Reich (2008) and extensively applied in financial portfolio construction by de Prado (2016), recovers data-driven groups suitable for use as the group structure in group LASSO.

The combination of sparsity, sign constraints, and cluster consistency, however, is not available as a single coherent software artifact. In the Python ecosystem, **scikit-learn**'s `Lasso` class (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, and Édouard Duchesnay 2011) handles the basic single-response LASSO with no group structure and no multi-response capability. The **group-lasso**, **asgl**, and **celer** packages handle group LASSO with externally specified groups but do not perform cluster discovery and do not implement sign constraints at the cell level. The **adelie** package (Yang and Hastie 2024) provides a fast multi-task group LASSO solver but again requires the group structure as an input. Beyond Python, the R packages **gglasso** (Yang and Zou 2015), **sparsegl** (Liang, Cohen, Sólon Heinsfeld, Pestilli, and McDonald 2024), and **oem** cover overlapping subsets of this surface. None of these packages combines cluster discovery, cluster-pooled sign derivation with a noise-floor gate, adaptive penalty reweighting routed through both the $\ell_1$ and group $\ell_2$ paths, and prior-centred regularisation toward a non-zero baseline. Financial-panel estimation requires all four together.

The **factorlasso** package, presented in this paper, supplies that combination. Specifically, **factorlasso** implements:

1. Multi-response sparse regression on the loading matrix $\beta \in \mathbb{R}^{N \times M}$ in a single convex programme, with the cross-asset information entering through a *precomputed* cluster-pooled sign matrix and adaptive penalty weights rather than through a cross-asset group norm. Hierarchical clustering group LASSO (HCGL) discovers the cluster structure by Ward linkage on the response correlation matrix and uses it to pool the sign derivation and the adaptive weights. Given the sign matrix and weights, the programme is block-separable across assets and reduces to $N$ per-asset second-order cone problems solved as one object.

2. Cluster-pooled univariate sign derivation: the univariate slope computed against the cluster-mean response and broadcast to every cluster member as a hard sign constraint, gated by a noise-floor $t$ statistic that pins weak-evidence cells to zero. A closed-form residual sum-of-squares (SSR) identity vectorises the gate across factors (Appendix A).

3. Adaptive penalty reweighting following Zou (2006) and Wang and Leng (2008), routed through both the $\ell_1$ and group $\ell_2$ paths and driven by the same pooled slope that produces the sign matrix.

4. Prior-centred regularisation, in which the penalty pulls toward an economically motivated $\beta^0 \neq 0$ rather than toward zero.

5. Consistent factor covariance assembly, in which the same $\beta$ matrix used in the conditional mean is reused in the implied covariance, producing alpha-beta consistency by construction (Sepp, Hansen, and Kastenholz (2026a)).

The contribution that gives **factorlasso** its name is the cluster layer driving the sign-derivation pooling and the cluster-size penalty weights. This shared layer is what differentiates the package from a configurable sparse-group LASSO solver such as **asgl** or **sparsegl**: the clusters are not external inputs but discovered from the data, and the same partition feeds the sign-derivation pooling, the cluster-size penalty weights, and the row-aggregation for adaptive reweighting. The convex programme itself uses a row-grouped $\ell_{2,1}$ penalty; the cluster enters it only through the per-cluster weight, not as the grouping unit of the norm.

The package is built on **CVXPY** (Diamond and Boyd 2016), using the modelling layer to express the sign constraints, prior centring, adaptive weights, and mixed-norm penalties as a single disciplined convex programme. This modelling language is widely used in convex-optimisation research and itself published in this journal as **CVXR** (Fu, Narasimhan, and Boyd 2020). When CVXPY verifies the problem as convex and the selected solver terminates with an optimal status, the fitted loading matrix is globally optimal up to solver tolerances. Section 3.3 details the solver choices, numerical safeguards, and retry policy used in the simulation harness. The cost of this formulation is additional modelling and conic-solver overhead relative to a bespoke coordinate-descent implementation. For the target financial-panel regime ($N \in [50, 200]$, $M \approx 10$, with monthly refits), the measured runtimes remain interactive, while the declarative formulation makes the constraint stack auditable and extensible.

The **factorlasso** package is the engine for factor-exposure estimation in the robust portfolio-optimisation framework of Sepp, Ossa, and Kastenholz (2026b), where the alpha-beta consistency of the covariance feeds the strategic and tactical allocation directly. It also estimates the capital market assumptions for strategic asset allocation in Sepp *et al.* (2026a).

The remainder of the paper is organised as follows. Section 2 develops the methodology: the multi-response factor model, the cluster-pooled sign derivation with its noise-floor gate, and the adaptive reweighting and prior-centring layers, with the closed-form SSR derivation and the sign-consistency proof sketch deferred to Appendices A and B. Section 3 documents the **factorlasso** software architecture, the core `LassoModel` estimator and its parameters, **scikit-learn** interoperability, and the in-repository simulation harness. Section 4 positions **factorlasso** against existing sparse-regression software. Section 5 summarises a simulation study in two parts: a synthetic ablation isolating where the cluster-pooled sign mechanism delivers gains, and a benchmark on a data-generating process calibrated to a multi-asset ETF universe, with the true loadings known, that pits the package against **scikit-learn**, **asgl**, and **skglm**. The full regime grid is reproduced by the scripts of Appendix C. Section 6 applies the package to that 102-fund multi-asset universe for capital-market-assumption estimation. Section 7 concludes.

## 2. Methodology

This section develops the cluster-pooled sign-derivation mechanism that distinguishes **factorlasso** from the existing sparse-regression ecosystem. Section 2.1 fixes notation for the multi-response factor model. Section 2.2 presents the pooled univariate slope, the closed-form SSR identity that vectorises the noise-floor $t$ statistic gate (full derivation in Appendix A), and the gate itself. Section 2.3 describes the hierarchical clustering layer whose discovered clusters drive the sign-derivation pooling, the adaptive-weight aggregation, and the cluster-size multipliers in the group penalty, which the package offers in a row-grouped and a cluster-by-factor form.

Section 2.4 states the sign-constrained sparse group LASSO problem that the package's CVXPY backend solves. Section 2.5 adds the Zou–Wang–Leng adaptive penalty reweighting, routed through both the $\ell_1$ and group $\ell_2$ paths. Section 2.6 covers prior-centred regularisation. Section 2.7 documents the overlay rule by which a practitioner-supplied sign matrix combines with the data-derived signs. Appendix B sketches the heuristic asymptotic argument for the mechanism under stylised conditions and gives a proof sketch.

### 2.1. Multi-response factor model setup

Let $\boldsymbol{Y} \in \mathbb{R}^{T\times N}$ be the panel of $N$ asset returns observed over $T$ time periods and $\boldsymbol{X} \in \mathbb{R}^{T\times M}$ the panel of $M$ factor returns observed over the same window. The multi-response factor model is

$$\boldsymbol{Y} \;=\; \boldsymbol{X}\,\boldsymbol{\beta}^{\top} + \boldsymbol{\varepsilon}, \tag{1}$$

where $\boldsymbol{\beta} \in \mathbb{R}^{N\times M}$ is the loading matrix the package estimates and $\boldsymbol{\varepsilon} \in \mathbb{R}^{T\times N}$ is the residual. In the financial application context, $\boldsymbol{Y}$ and $\boldsymbol{X}$ contain demeaned and optionally EWMA-weighted asset returns and factor returns, respectively. The target estimate $\hat{\boldsymbol{\beta}}$ enters two downstream quantities: the implied per-asset factor risk premium $\hat{\boldsymbol{\beta}}\boldsymbol{\lambda}$ (where $\boldsymbol{\lambda} \in \mathbb{R}^{M}$ is the vector of factor risk premia) and the asset return covariance $\hat{\boldsymbol{\beta}}\,\boldsymbol{\Sigma}_F\,\hat{\boldsymbol{\beta}}^{\top} + \boldsymbol{D}$ (where $\boldsymbol{\Sigma}_F$ is the $M \times M$ factor covariance and $\boldsymbol{D}$ is a diagonal residual covariance). Using the same $\hat{\boldsymbol{\beta}}$ in

both quantities is what Sepp *et al.* (2026a) call alpha-beta consistency by construction. The term denotes internal consistency: the conditional mean $\hat{\boldsymbol{\mu}} = \hat{\boldsymbol{\alpha}} + \hat{\boldsymbol{\beta}}\boldsymbol{\lambda}$ and the covariance $\hat{\boldsymbol{\beta}}\boldsymbol{\Sigma}_F\hat{\boldsymbol{\beta}}^\top + \boldsymbol{D}$ are built from one fitted loading matrix, so a downstream optimiser sees a single coherent set of exposures rather than a mean and covariance estimated separately. Here $\hat{\boldsymbol{\alpha}} \in \mathbb{R}^N$ is the per-asset intercept, which the estimator recovers in original return units from the demeaning step of Equation 1 (Section 3.1, the `alpha_const_` attribute).

The EWMA weighting overweights recent observations, so the loadings track slow changes in factor exposures rather than averaging equally over a decade of returns, the standard treatment of time-varying second moments in financial-return data (Engle 2002). Zakamulin (2015) shows empirically that minimum-variance portfolios constructed from EWMA-weighted covariance matrices track their risk targets better than those constructed from the shrinkage estimator of Ledoit and Wolf (2004).

We denote by $\boldsymbol{x}_j \in \mathbb{R}^T$ the $j$-th factor column of $\boldsymbol{X}$, by $\boldsymbol{y}_k \in \mathbb{R}^T$ the $k$-th asset column of $\boldsymbol{Y}$, and by $\beta_{kj}$ the $(k, j)$ entry of $\boldsymbol{\beta}$. Asset-cluster partitions are written $\{\mathcal{C}_1, \ldots, \mathcal{C}_G\}$, where each $\mathcal{C}_g \subseteq \{1, \ldots, N\}$ is the set of asset indices in cluster $g$ and $|\mathcal{C}_g|$ its cardinality. We write $\mathcal{C}(k)$ for the (unique) cluster containing asset $k$.

## 2.2. Univariate slopes and the SSR identity

**Pooled univariate slope.** For each factor $j$, the pooled univariate slope across all $N$ assets satisfies the no-intercept OLS first-order condition $\boldsymbol{x}_j^\top \sum_{k=1}^N (\boldsymbol{y}_k - \hat{\beta}_j \boldsymbol{x}_j) = 0$, which gives

$$\hat{\beta}_j \;=\; \frac{\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}}}{N\, \boldsymbol{x}_j^\top \boldsymbol{x}_j}, \qquad \boldsymbol{y}_{\text{sum}} \;\equiv\; \sum_{k=1}^{N} \boldsymbol{y}_k \;\in\; \mathbb{R}^T. \tag{2}$$

Equation 2 is the multi-response generalisation of the single-response univariate slope formula of Chatterjee *et al.* (2025), with the cross-asset sum $\boldsymbol{y}_{\text{sum}}$ playing the role of the response.

When the assets are grouped into clusters $\{\mathcal{C}_1, \ldots, \mathcal{C}_G\}$, the slope is computed against the cluster-sum response and broadcast to every cluster member,

$$\hat{\beta}_{j,\mathcal{C}} \;=\; \frac{\boldsymbol{x}_j^\top \boldsymbol{y}_{\mathcal{C},\text{sum}}}{|\mathcal{C}|\, \boldsymbol{x}_j^\top \boldsymbol{x}_j}, \qquad \boldsymbol{y}_{\mathcal{C},\text{sum}} \;=\; \sum_{k \in \mathcal{C}} \boldsymbol{y}_k \;\in\; \mathbb{R}^T. \tag{3}$$

This is the natural cluster restriction of Equation 2: replacing $\{1, \ldots, N\}$ with the cluster index set $\mathcal{C}$ and $N$ with $|\mathcal{C}|$ recovers Equation 2 when $\mathcal{C} = \{1, \ldots, N\}$. Equivalently, the cluster-pooled slope is the OLS slope of the cluster-mean response $\bar{\boldsymbol{y}}_{\mathcal{C}} = |\mathcal{C}|^{-1}\boldsymbol{y}_{\mathcal{C},\text{sum}}$ on $\boldsymbol{x}_j$. The cluster-pooled slope produces within-cluster sign coherence by construction: when two assets in the same cluster have a high pairwise correlation in $\boldsymbol{Y}$ (say, two investment-grade credit indices with empirical corr $> 0.95$), the pooled slope produces a single sign decision shared by both, eliminating the spurious within-cluster long-short alternations that an unconstrained LASSO would produce when minimising in-sample residual variance.

**Closed-form SSR identity and noise-floor gate.** To filter sign constraints driven by sampling noise rather than economic signal, the univariate $t$ statistic is computed from a closed-form residual sum-of-squares identity that avoids materialising the per-cell residuals,

$$\text{SSR}_j \;=\; \|\boldsymbol{Y}\|_F^2 \;-\; N\, \hat{\beta}_j^2\, (\boldsymbol{x}_j^\top \boldsymbol{x}_j), \tag{4}$$

which follows from the slope-defining identity $\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}} = N\,\hat{\beta}_j(\boldsymbol{x}_j^\top \boldsymbol{x}_j)$ (derivation, the missing-observation accounting, and the cluster-pooled variant in Appendix A). Equation 4 is vectorisable over $j$ in one matrix product, so the entire $(N, M)$ sign matrix is computed in a single Python call. The empirical speedup over a residual-materialisation baseline is one to two orders of magnitude across the panel sizes typical of multi-asset estimation (benchmark in `benchmarks/ssr_speedup.py`). The pooled variance, standard error, and $t$ statistic $t_j$ follow in closed form from $\text{SSR}_j$, and the cluster-pooled $t_{j,\mathcal{C}}$ substitutes $|\mathcal{C}|$ for $N$ and restricts the Frobenius norm to the cluster columns (Appendix A). The gated sign matrix consumed by the multivariate solver is

$$s_{kj} \;=\; \begin{cases} \operatorname{sign}(\hat{\beta}_{j,\mathcal{C}(k)}), & |t_{j,\mathcal{C}(k)}| \;\geq\; \tau, \\ 0, & |t_{j,\mathcal{C}(k)}| \;<\; \tau, \end{cases} \tag{5}$$

where $\mathcal{C}(k)$ is the cluster containing asset $k$ and $\tau$ is the `auto_sign_threshold_t` hyperparameter (package default $\tau = 0.75$). The threshold is not a classical significance test (the pooled slopes have no exact null distribution here). The threshold is a defensive filter that removes the worst noise-driven sign constraints while preserving moderate-evidence signals, screening marginal evidence before regularising. The gate operates at cluster granularity: when $|t_{j,\mathcal{C}}| < \tau$ it sets $s_{kj} = 0$ for every member $k \in \mathcal{C}$, forcing $\beta_{kj} = 0$ for the whole cluster on factor $j$ and preserving within-cluster sign coherence as a strict invariant. When strong prior economic evidence is available, the practitioner overlay of Section 2.7 can pierce a cluster decision at the per-cell level.

### 2.3. Hierarchical clustering group LASSO

The cluster partition $\{\mathcal{C}_1, \ldots, \mathcal{C}_G\}$ that feeds Equation 3 can be supplied externally (via the `group_data` argument when an asset-class taxonomy is known) or discovered from the data. **factorlasso** discovers it by Ward linkage on the response correlation matrix.

Let $\boldsymbol{C} = \operatorname{corr}(\boldsymbol{Y})$ denote the $N \times N$ Pearson correlation matrix of the asset returns. We form the correlation dissimilarity $\boldsymbol{D} = \mathbf{1} - \boldsymbol{C}$ and apply Ward agglomerative linkage to squareform($\boldsymbol{D}$), then cut the dendrogram at height $\rho_{\text{cut}} \cdot \max_{i,j} D_{ij}$, where $\rho_{\text{cut}} \in (0, 1]$ is the `cutoff_fraction` hyperparameter (default 0.5). We use $1 - \rho_{ij}$ as the dissimilarity rather than the Euclidean chord distance $\sqrt{2(1-\rho_{ij})}$ between standardised return vectors. The two agree up to a concave monotone transform, so we read this step as a stable correlation-clustering heuristic rather than as exact Ward variance minimisation in Euclidean space.[1] Ward linkage is the default because it is markedly more stable than single linkage under correlated, high-variance features (the regime of asset-return panels), a contrast documented across the synthetic clustering experiments of Sorooshyari, Rivas, and Tibshirani (2026). The discovered partition has between one cluster (at $\rho_{\text{cut}} \to 1$) and $N$ singleton clusters (at $\rho_{\text{cut}} \to 0$) and is determined by the data, not chosen by the practitioner.

The same partition serves three purposes inside the estimator: as the source of the per-cluster weight $\sqrt{|\mathcal{C}_g|/G}$ on the row-grouped penalty in Equation 6, as the asset-pooling unit for the sign-derivation slope in Equation 3, and as the asset-pooling unit for the row aggregation in the adaptive group penalty of Equation 11. Using a single partition across all three layers is what

[1] Ward's criterion is derived for Euclidean distances between points, so under $1 - \rho$ the merge costs are not exact within-cluster sum-of-squares increments. We retain $1 - \rho$ because the partition is used only to pool signs and to group the penalty, not to make a metric claim.

makes **factorlasso** a structurally coherent estimator rather than a collection of independently-tunable components.

### 2.4. Sign-constrained sparse group LASSO

The sign matrix $\boldsymbol{S} = (s_{kj}) \in \{-1, 0, +1, \mathrm{NaN}\}^{N \times M}$ from Equation 5 is consumed by the prior-centred multi-response sparse group LASSO problem

$$\hat{\boldsymbol{\beta}} \;=\; \arg\min_{\boldsymbol{\beta} \in \mathcal{K}(\boldsymbol{S})} \left\{ \frac{1}{T} \sum_{t=1}^{T} w_t \left\| \boldsymbol{y}_t - \boldsymbol{\beta}\, \boldsymbol{x}_t \right\|_2^2 \;+\; \lambda\,(1-\alpha) \sum_{g=1}^{G} \sqrt{|\mathcal{C}_g|/G} \sum_{k \in \mathcal{C}_g} \left\| \boldsymbol{\beta}_{k,\cdot} - \boldsymbol{\beta}^0_{k,\cdot} \right\|_2 \right.$$
$$\left. \;+\; \lambda\,\alpha \sum_{k=1}^{N} \sum_{j=1}^{M} |\beta_{kj} - \beta^0_{kj}| \right\}, \tag{6}$$

$$\mathcal{K}(\boldsymbol{S}) = \{\boldsymbol{\beta} : \beta_{kj} = 0 \text{ if } s_{kj} = 0, \quad s_{kj}\,\beta_{kj} \geq 0 \text{ if } s_{kj} \in \{-1, +1\}\}\,. \tag{7}$$

Cells with $s_{kj} = \mathrm{NaN}$ are omitted from $\mathcal{K}(\boldsymbol{S})$ and are therefore unconstrained.

Here $w_t$ are non-negative observation weights (uniform by default, EWMA-decayed when the `span` parameter is set), $\boldsymbol{\beta}^0 \in \mathbb{R}^{N \times M}$ is an optional prior loading matrix (zero by default, an economically-motivated value when supplied), $\lambda > 0$ is the regularisation strength (`reg_lambda`), and $\alpha \in [0, 1]$ is the sparse-group mixing parameter (`l1_weight`, where $\alpha = 0$ recovers pure group LASSO and $\alpha = 1$ recovers pure LASSO at the cell level). The group penalty has a row-grouped $\ell_{2,1}$ mixed-norm form: a per-asset $\ell_2$ norm on the loading vector $\beta_{k,\cdot} \in \mathbb{R}^M$, summed across assets and scaled by a cluster-size multiplier. The mathematical groups selected by the norm are the asset rows; the HCGL partition enters through the common multiplier applied to rows in the same cluster, and through the sign-derivation and adaptive-weighting steps. The normalised multiplier $\sqrt{|C_g|/G}$ is a heuristic cluster-size scaling used to moderate the relative influence of large clusters. This scaling is not invariant to arbitrary refinements of the partition. The unnormalised cluster-size multiplier $\sqrt{|\mathcal{C}_g|}$ (Yuan and Lin 2006) is available via the `group_penalty` parameter.

Equation 6 groups the penalty along the rows of the loading matrix: the $\ell_2$ norm runs over the $M$ factor loadings of one asset, and the cluster enters only through the cluster-size multiplier. **factorlasso** also offers the complementary grouping, in which the $\ell_2$ norm runs over the assets of a cluster on one factor. We call the first mode hierarchical-clustering group LASSO (HCGL, `HIERARCHICAL_CLUSTER_GROUP_LASSO`) and the second factor-clustering group LASSO (FCGL, `FACTOR_CLUSTER_GROUP_LASSO`). The FCGL group penalty replaces the per-row norm with a per-block norm over each cluster-by-factor block,

$$\lambda\,(1-\alpha) \sum_{g=1}^{G} \sqrt{|\mathcal{C}_g|/G} \sum_{j=1}^{M} \left\| \boldsymbol{\beta}_{\mathcal{C}_g,j} - \boldsymbol{\beta}^0_{\mathcal{C}_g,j} \right\|_2, \tag{8}$$

where $\boldsymbol{\beta}_{\mathcal{C}_g,j} \in \mathbb{R}^{|\mathcal{C}_g|}$ collects the loadings of the cluster's assets on factor $j$. In FCGL the cluster is the group selected by the norm itself, not only the multiplier, so a whole cluster-by-factor block enters or leaves the model together. The two modes encode different beliefs about where loadings are sparse. HCGL treats each asset as the unit and lets the loadings vary freely within a cluster, which fits a heterogeneous cluster. FCGL shrinks a cluster's loadings on a factor jointly toward the prior through the per-block norm, which fits a homogeneous cluster.

When the shrinkage binds fully the block collapses to the prior. When the cluster carries a strong shared signal the block retains loadings away from the prior, shrunk as a group rather than equalised. We present both as alternatives and characterise their behaviour in Section 5 rather than designating one as preferred; the appropriate choice depends on the within-cluster homogeneity of the application.

The sign-constraint set 7 is enforced as a hard convex constraint in the CVXPY problem (Diamond and Boyd 2016). Cells with $s_{kj} = 0$ are forced to $\beta_{kj} = 0$. Cells with $s_{kj} \in \{-1, +1\}$ are constrained to the matching half-space. Cells with $s_{kj} = \text{NaN}$ are unconstrained. Together with the $\ell_1$ and group $\ell_2$ penalty terms, the constraint set is convex and the entire problem reduces to a second-order cone programme solvable by CLARABEL, ECOS, or SCS via CVXPY's standard cone-solver interface.

The sign matrix $\boldsymbol{S}$ and the adaptive weights of Section 2.5 are computed before the cone solve. Once they are fixed, every term in Equation 6 decomposes across the asset index $k$: the weighted fit is a sum of per-response squared errors, the group penalty is a sum of per-row $\ell_2$ norms scaled by the constant cluster weight $\sqrt{|\mathcal{C}_g|/G}$, and the $\ell_1$ term and sign constraints act cell by cell. The programme is therefore block-separable across assets and is statistically identical to $N$ per-asset fits under the shared sign matrix and weights. The cross-asset information enters through those precomputed quantities, not through a cross-asset group norm. **factorlasso** solves the $N$ blocks as one object for interface and bookkeeping convenience (one `fit` call, one fitted $\boldsymbol{\beta}$, one serialisable covariance), at the runtime cost documented in Section 3.3. The block-separability is a property of HCGL only. The FCGL penalty of Equation 8 couples the assets of a cluster through the per-block norm, so the FCGL programme does not decompose into per-asset fits and is solved as one coupled cone programme.

## 2.5. Adaptive penalty reweighting

When the practitioner sets `auto_sign_adaptive_weights = True`, the penalty terms in Equation 6 are replaced by magnitude-aware reweighted versions. The per-cell weight matrix follows Zou (2006):

$$W_{kj} \;=\; \frac{1}{\max\big(|\hat{\beta}_{j,\mathcal{C}(k)}|,\, \varepsilon\big)^{\gamma}}, \tag{9}$$

where $\hat{\beta}_{j,\mathcal{C}(k)}$ is the cluster-pooled slope from Equation 3, $\gamma$ is the Zou exponent (default $\gamma = 1$, parameter `auto_sign_adaptive_gamma`), and $\varepsilon > 0$ is a stabiliser (default $\varepsilon = 10^{-3}$, parameter `auto_sign_adaptive_floor`) preventing weight explosion on near-zero slopes. Cells where the sign gate of Equation 5 has set $s_{kj} = 0$ receive the placeholder $W_{kj} = 1$. The hard sign constraint forces $\beta_{kj} = 0$ regardless, so the penalty contribution at those cells is identically zero. These cell weights enter the optimisation problem along two distinct routes, both controlled by the same boolean flag.

The first route is the cellwise $\ell_1$ path, active when the sparse-group mixing weight $\alpha$ in Equation 6 is non-zero. The $\ell_1$ term is replaced by a Zou-reweighted variant

$$\lambda\, \alpha \sum_{k,j} W_{kj}\, |\beta_{kj} - \beta^0_{kj}|. \tag{10}$$

The second route is the group $\ell_2$ path, which is the route that activates in the pure-group-LASSO production configuration where the $\ell_1$ mixing weight is zero. Following the adaptive

group LASSO of Wang and Leng (2008), the per-cell weights are aggregated per-asset by root-mean-square over the non-pinned factors,

$$W_k \;=\; \sqrt{\frac{1}{|\mathcal{J}_k|}\sum_{j\in\mathcal{J}_k} W_{kj}^2}, \qquad \mathcal{J}_k \;=\; \{\, j : s_{kj} \neq 0 \,\}, \tag{11}$$

and each asset's contribution to the group penalty is scaled by its row weight:

$$\lambda\,(1-\alpha)\sum_{g=1}^{G}\sqrt{|\mathcal{C}_g|/G}\sum_{k\in\mathcal{C}_g} W_k\,\|\boldsymbol{\beta}_{k,\cdot} - \boldsymbol{\beta}^0_{k,\cdot}\|_2. \tag{12}$$

The root-mean-square aggregation is the $\ell_2$-natural counterpart to the $\ell_2$ norm in the group penalty: for an asset with $|\hat{\beta}_{j,\mathcal{C}}| = 1$ uniformly across its non-pinned factors, Equation 11 gives $W_k = 1$ exactly, preserving the existing per-cluster scaling $\sqrt{|\mathcal{C}_g|/G}$. Assets with all cells pinned by the gate (the degenerate $\mathcal{J}_k = \emptyset$ case) fall back to $W_k = 1$.

The default `auto_sign_adaptive_weights = False` preserves the uniform-penalty formulation of Equation 6. The reweighting is independent of the threshold gate of Section 2.2 and composes naturally with it: strong-evidence factors (large $|\hat{\beta}_{j,\mathcal{C}}|$) receive a lighter penalty and can take larger multivariate coefficients without artificial shrinkage. Weak-evidence factors receive a heavier penalty and are pushed harder toward the prior. The reweighting is continuous and magnitude-aware where the threshold gate is binary. The two mechanisms address complementary aspects of the same noise-versus-signal trade-off.

### 2.6. Prior-centred regularisation

The penalty terms in Equations 6, 10, and 12 are written as shrinkage toward an optional prior loading matrix $\boldsymbol{\beta}^0 \in \mathbb{R}^{N\times M}$. When $\boldsymbol{\beta}^0 = \mathbf{0}$ (the default, when the `factors_beta_prior` argument is left unset) the formulation reduces to the standard zero-centred sparse group LASSO. When $\beta^0$ is populated with economically motivated values, the penalty shrinks the fit toward those values rather than toward zero. As $\lambda \to 0$, the estimator approaches the constrained weighted least-squares solution; as $\lambda$ increases, the solution is pulled toward the feasible part of $\beta^0$. The cellwise $\ell_1$ path has the usual Laplace-prior interpretation, while the unsquared row $\ell_2$ path corresponds to a group-Laplace prior centred at $\beta^0$.

Prior-centred shrinkage toward $\boldsymbol{\beta}^0 \neq \mathbf{0}$ is the maximum-a-posteriori estimator under a Laplace prior centred at $\boldsymbol{\beta}^0$. In portfolio optimisation, it is the estimation analogue of the idea at the centre of the Black–Litterman model (Black and Litterman 1992): blend an economically motivated target with the information in the data rather than estimate from the data alone. Black–Litterman blends a prior on expected returns with market-implied equilibrium returns. **factorlasso** blends a prior on factor *loadings* with the sample fit, at the cell level and subject to sign constraints. The covariance assembly of Section 3.4, $\boldsymbol{\beta}\boldsymbol{\Sigma}_F\boldsymbol{\beta}^\top + \boldsymbol{D}$, is a structured shrinkage estimator of the asset covariance in the tradition of Ledoit and Wolf (2004): the factor structure plays the role of the shrinkage target and the diagonal residual term the role of the sample-driven component. The contribution of **factorlasso** is not the prior-shrinkage idea, which is standard, but the cell-level combination of a non-zero prior with hard sign constraints and cluster-pooled sign derivation inside one estimator (Section 4).

This is the mechanism by which the Multi-Asset Tradable Factors (MATF)-CMA framework of Sepp *et al.* (2026a) injects non-zero structural priors for asset classes whose factor exposures

are known a priori but would otherwise be eliminated by the $\ell_1$ or group $\ell_2$ penalty (e.g., a positive credit-beta prior for high-yield assets even when sample correlation with the credit factor is below the noise floor). Sign constraints take precedence over prior pulls: if a prior $\beta^0_{kj}$ has sign opposite to the derived $s_{kj}$, the constraint set blocks the optimiser from moving $\hat{\beta}_{kj}$ toward $\beta^0_{kj}$. When the data term does not pull the coefficient into the interior of the permitted half-space, the optimiser settles at $\hat{\beta}_{kj} = 0$, the boundary of the constraint set.

### 2.7. Overlay with practitioner-supplied signs

When the practitioner supplies an explicit per-cell sign matrix $\boldsymbol{S}^{\text{exp}} \in \{-1, 0, +1, \text{NaN}\}^{N \times M}$ via the `factors_beta_loading_signs` argument alongside `auto_sign_constraints = True`, the final sign matrix consumed by the solver combines the two by NaN-overlay:

$$s_{kj} = \begin{cases} S^{\text{exp}}_{kj}, & S^{\text{exp}}_{kj} \neq \text{NaN}, \\ \text{data-derived from Equation 5}, & \text{otherwise.} \end{cases} \tag{13}$$

The practitioner override always wins. This overlay is the mechanism by which the structural sign matrix of Sepp *et al.* (2026a) (their Equation 13) is enforced: private equity loadings are forced to zero for traditional asset classes, rates volatility loadings are constrained non-positive for corporate credit assets, and the remaining cells inherit data-derived signs from the auto-derivation pass. This preserves the asset-class economic identity that pure marginal correlation cannot identify while delegating the within-class differentiation to the data.

A heuristic asymptotic argument for the mechanism, with the formal support-sign-consistency statement and a proof sketch, is given in Appendix B.

## 3. Software architecture

The **factorlasso** package exposes a single core estimator, `LassoModel`, that wraps the methodology of Section 2 behind a uniform constructor-and-`fit` interface compatible with the **scikit-learn** (Pedregosa *et al.* 2011) convention. This section documents the estimator's parameters (Section 3.1), the dedicated data-derived sign-constraint parameter family (Section 3.2), the CVXPY backend choice and solver fallback chain (Section 3.3), the downstream factor-covariance assembly classes (Section 3.4), and a minimal end-to-end usage example (Section 3.5). The simulation harness shipped alongside the package for reproducing the methodology study is documented in Appendix C.

### 3.1. The LassoModel estimator

The estimator is constructed by passing hyperparameters to `LassoModel(...)` and fitted by calling `.fit(x, y)` on factor and asset return panels. Three `model_type` values select the underlying optimisation problem:

- `LassoModelType.LASSO` solves the cellwise $\ell_1$-penalised problem with no group structure. This mode is the baseline used for ablation comparisons in Section 5.
- `LassoModelType.GROUP_LASSO` solves the group-penalised problem with an externally supplied partition via the `group_data` argument. The partition is typically a known asset-class taxonomy.

| Parameter | Default | Purpose |
|---|---|---|
| `model_type` | `LASSO` | Selects the optimisation problem |
| `reg_lambda` | $10^{-5}$ | Regularisation strength $\lambda$ |
| `l1_weight` | 0 | Sparse-group mixing $\alpha \in [0, 1]$ |
| `group_penalty` | `"normalized"` | Group weight, normalised or Yuan–Lin |
| `cutoff_fraction` | 0.5 | HCGL dendrogram cut as fraction of max $D_{ij}$ |
| `group_data` | `None` | External group partition for `GROUP_LASSO` |
| `factors_beta_loading_signs` | `None` | Practitioner sign overlay $\boldsymbol{S}^{\text{exp}}$ |
| `factors_beta_prior` | `None` | Prior loading matrix $\boldsymbol{\beta}^0$ |
| `auto_sign_constraints` | `False` | Enable cluster-pooled sign derivation |
| `auto_sign_threshold_t` | 0.75 | Noise-floor gate $\tau$ |
| `auto_sign_adaptive_weights` | `False` | Enable adaptive penalty reweighting |
| `auto_sign_adaptive_gamma` | 1 | Adaptive weight exponent $\gamma$ |
| `auto_sign_adaptive_floor` | $10^{-3}$ | Stabiliser $\varepsilon$ in Equation 9 |
| `span` | `None` | EWMA span for observation weights |
| `span_freq_dict` | `None` | Per-frequency span override |
| `warmup_period` | 12 | Minimum valid observations per asset; loadings of shorter-history assets are set to zero |
| `demean` | `True` | Center $\boldsymbol{X}$ and $\boldsymbol{Y}$ before fitting |
| `nonneg` | `False` | Global non-negativity constraint on all $\beta_{kj}$ |
| `solver` | `"CLARABEL"` | Primary CVXPY cone solver |

Table 1: Constructor parameters of the **factorlasso** `LassoModel` estimator and their package defaults. The calibrated benchmark of Section 5.5 and the application of Section 6 run a fixed *production configuration* of these parameters, the parameter preset of the deployed multi-asset estimator, stated in Section 5.5. The sign-derivation parameter family is documented in Section 3.2.

- `LassoModelType.HIERARCHICAL_CLUSTER_GROUP_LASSO` solves the group-penalised problem with the partition discovered internally by HCGL on corr($\boldsymbol{Y}$), as described in Section 2.3.
- `LassoModelType.FACTOR_CLUSTER_GROUP_LASSO` solves the FCGL problem of Equation 8 on the same discovered partition, grouping the penalty over each cluster-by-factor block rather than over each asset row.

Beyond the model-type selector, the constructor takes parameters across five categories: regularisation strength and mixing (`reg_lambda`, `l1_weight`, `group_penalty`), cluster discovery (`cutoff_fraction`), data-derived sign constraints (the `auto_sign_*` family, Section 3.2), time decay (`span`, `span_freq_dict`, `warmup_period`), and solver selection (`solver`, `demean`, `nonneg`). The practitioner-overlay sign matrix and the prior loading matrix are supplied via `factors_beta_loading_signs` and `factors_beta_prior` respectively. Table 1 summarises the constructor parameters and their default values.

The default `reg_lambda` of $10^{-5}$ is deliberately near zero, so the out-of-the-box fit is essentially unregularised multi-response least squares. We set it this way because the regularisation strength is problem-specific and is meant to be chosen by cross-validation or along the $\lambda$-path (Section 3.5), not assumed: a non-trivial default would shrink silently before the user has selected a value. The deployed configuration sets `reg_lambda` explicitly.

After fitting, the estimator exposes twelve attributes following the **scikit-learn** trailing-underscore convention. The primary outputs are `coef_` (the $N \times M$ fitted $\hat{\boldsymbol{\beta}}$ matrix as a **pandas** data frame with asset names as index and factor names as columns), `intercept_` (the per-asset constant term), `derived_signs_` (the $N \times M$ sign matrix from Equation 5, set when `auto_sign_constraints` is `True`), `clusters_` (the discovered cluster partition as a **pandas** series), and `estimation_result_` (a dataclass carrying sum-of-squares decomposition and $R^2$). Diagnostic attributes `linkage_` and `cutoff_` carry the HCGL dendrogram and the cut distance, `valid_mask_` flags non-`NaN` cells in the input panel, `effective_span_` carries the EWMA decay used, `x_` and `y_` hold the fit inputs, and `alpha_const_` holds the economic intercept of the regression in original units, paired consistently with the fitted loadings.

### 3.2. Data-derived sign constraints: the sign-derivation parameter family

The package's headline contribution surface is the family of five parameters governing data-derived sign constraints. Each parameter maps directly to one element of the methodology of Section 2, and the parameters are designed to compose: turning on `auto_sign_constraints` alone gives the noise-floor gated sign mechanism of Section 2.2. Additionally turning on `auto_sign_adaptive_weights` adds the Zou–Wang–Leng reweighting of Section 2.5.

`auto_sign_constraints` (boolean, default `False`) is the master switch. When set to `True`, `LassoModel.fit` computes the pooled univariate slope from Equation 3 on the cluster-sum response for each cluster, computes the closed-form $t$ statistic from Equations 4 and 15, applies the noise-floor gate from Equation 5, and broadcasts the resulting cluster-level sign decision to every cluster member. The sign matrix produced by the pass is enforced as a hard convex constraint inside the CVXPY problem of Equation 6 and is also persisted on the fitted estimator as `derived_signs_` for downstream auditing and the factor-covariance dataclass field of Section 3.4.

`auto_sign_threshold_t` (float, default 0.75) is the $\tau$ parameter of Equation 5. The package default is 0.75 and is held constant across the synthetic ablation regimes of Section 5. The deployed configuration runs the stricter $\tau = 1.0$, the value used in the calibrated benchmark of Section 5.5 and the application of Section 6. Increasing $\tau$ pins more cells to zero (stricter gate, fewer signs constraining the fit). Decreasing $\tau$ admits more signs into the constraint set (looser gate, more signs constraining the fit). The parameter is decoupled from the regularisation strength $\lambda$ by design: $\tau$ governs which cells are eligible for sign-constraint participation, $\lambda$ governs the shrinkage of cells that are eligible.

`auto_sign_adaptive_weights` (boolean, default `False`) activates the Zou–Wang–Leng adaptive reweighting of Section 2.5. The same boolean controls both the cellwise $\ell_1$ path (Equation 10) and the group $\ell_2$ path (Equation 12). Which one is dominant depends on the value of `l1_weight`. The default `False` preserves the uniform-penalty formulation of Equation 6.

`auto_sign_adaptive_gamma` (float, default 1) is the Zou exponent $\gamma$ in Equation 9. The default of $\gamma = 1$ recovers the canonical adaptive LASSO weight (Zou 2006). Values closer to zero reduce the magnitude-aware differentiation. Values above one amplify it.

`auto_sign_adaptive_floor` (float, default $10^{-3}$) is the stabiliser $\varepsilon$ in Equation 9 that prevents the weight $W_{kj}$ from exploding on near-zero pooled slopes. The default has been chosen so that even the smallest pooled slope in the financial-panel regime produces a weight bounded above by $10^3$, well within numerical tolerance of the CVXPY solver chain.

The parameter family is composable. Disabling `auto_sign_constraints` disables the gate, the adaptive reweighting, and the constraint set in one operation, recovering the standard sparse group LASSO of Simon *et al.* (2013) extended only by HCGL group discovery and EWMA observation weighting.

### 3.3. CVXPY backend and solver fallback chain

**factorlasso** expresses the optimisation problem of Equation 6 as a CVXPY (Diamond and Boyd 2016) disciplined convex programme and delegates the solve to one of the conic solvers in CVXPY's solver chain. The choice of CVXPY as the modelling layer is deliberate. Each constraint and penalty term that the methodology of Section 2 adds (sign overlay, adaptive penalty weights, prior centring, group structure) is expressed declaratively as a CVXPY term, and the underlying solver guarantees convergence to the global optimum for any disciplined convex programme (Grant, Boyd, and Ye 2006) up to solver tolerances. This formulation is the same architectural choice that Fu *et al.* (2020) made for the **CVXR** package published in this journal: the methodology contribution lives in the problem formulation, and the modelling layer is the natural place to express, audit, and extend it.

The trade-off is a modest constant-factor overhead relative to a hand-rolled coordinate-descent solver such as the one in **sparsegl** (Liang *et al.* 2024) or **adelie** (Yang and Hastie 2024). In the financial-panel regime ($N \in [50, 200]$, $M \approx 10$, quarterly refits) the **factorlasso** fit completes in seconds even with the full `auto_sign_*` stack engaged, and the overhead relative to an unconstrained CVXPY group LASSO is approximately 20 percent in rolling-window calibration.

The primary solver is CLARABEL (Goulart and Chen 2024), an interior-point solver shipped with CVXPY by default. When CLARABEL encounters numerical pathologies on a specific problem instance (rare in practice but observed in our simulation grid in roughly one in several thousand fits, exclusively on the orthogonal factor regime), the installed package raises the CVXPY solver error to the caller rather than silently masking it. The caller then retries with a different solver through the `solver` argument (CVXPY exposes ECOS and SCS among others). The package itself runs no automatic fallback chain. The simulation harness wraps the fit to retry with ECOS and then SCS, to keep the study grid 100 percent complete, and logs the solver used per cell so that any non-CLARABEL fit can be audited.

### 3.4. Factor covariance assembly

The fitted $\hat{\boldsymbol{\beta}}$ matrix is the primary input to the downstream factor covariance assembly that produces alpha-beta consistent CMA estimates of the form $\hat{\boldsymbol{\beta}}\,\boldsymbol{\Sigma}_F\,\hat{\boldsymbol{\beta}}^\top + \boldsymbol{D}$. The package provides two dataclasses for this purpose: `CurrentFactorCovarData` for a single-window estimate, and `RollingFactorCovarData` for a sequence of estimates over a rolling window.

`CurrentFactorCovarData` holds the fitted loading matrix $\hat{\boldsymbol{\beta}}$, the factor covariance $\boldsymbol{\Sigma}_F$, the residual covariance $\boldsymbol{D}$, the per-asset $R^2$ decomposition, the discovered cluster partition, and the derived sign matrix as the `derived_signs` field. The dataclass supports filtering by ticker

subset (`filter_on_tickers`), persistence to and from `xlsx` files (`save` and `load`), and round-trip identity preservation: filtering and then loading from disk produces an object bit-identical to the in-memory instance.

`RollingFactorCovarData` is a wrapper over a sequence of `CurrentFactorCovarData` instances indexed by the rolling-window end-date. This container is the structure consumed by the deployed portfolio-optimisation and CMA estimation pipeline, where each month or quarter-end produces one new entry and the historical entries provide the input to model-risk validation and consistency tests across rebalancing cycles.

The estimator admits assets whose returns are sampled at different frequencies. Liquid instruments report monthly, while illiquid sleeves such as private equity and private credit report quarterly. The per-frequency `span_freq_dict` argument of Table 1 sets a separate exponential-weighting span for each reporting frequency, and the estimator demeans the panel column by column on each asset's valid observations, so a quarterly sleeve with a short history neither dilutes the monthly cross-section nor forces the whole panel down to the coarsest common frequency. The resulting per-frequency fits feed the downstream covariance assembly of the **optimalportfolios** package (Sepp 2024), which assembles the estimated loadings into a rolling sequence indexed by rebalancing date.

### 3.5. A minimal usage example

The following code fits the production configuration of **factorlasso** on a synthetic panel and inspects the key fitted attributes.

```
>>> import numpy as np
>>> import pandas as pd
>>> from factorlasso import LassoModel, LassoModelType
>>>
>>> rng = np.random.default_rng(0)
>>> T, N, M = 120, 50, 9
>>> X = pd.DataFrame(rng.standard_normal((T, M)),
...                  columns=[f"F{j}" for j in range(M)])
>>> beta_true = rng.standard_normal((N, M)) * 0.4
>>> Y = pd.DataFrame(X.values @ beta_true.T +
...                  0.3 * rng.standard_normal((T, N)),
...                  columns=[f"A{k}" for k in range(N)])
>>>
>>> model = LassoModel(
...     model_type=LassoModelType.HIERARCHICAL_CLUSTER_GROUP_LASSO,
...     reg_lambda=1e-3,
...     cutoff_fraction=0.5,
...     auto_sign_constraints=True,
...     auto_sign_threshold_t=0.75,
...     auto_sign_adaptive_weights=True,
... ).fit(x=X, y=Y)
```

After fitting, the loading matrix, the derived sign matrix, and the discovered cluster partition are available as attributes:

```
>>> model.coef_.shape
>>> model.derived_signs_.shape
>>> int(model.clusters_.nunique())

(50, 9)
(50, 9)
12
```

The fitted estimator can be passed to the factor covariance assembly to produce a `CurrentFactorCovarData` instance, persisted to `xlsx`, and reloaded into an identical object. The full deployed pipeline that produces the covariance matrix and CMAs follows this same pattern with the additional inputs of the practitioner sign overlay $\boldsymbol{S}^{\exp}$ and the prior loading matrix $\boldsymbol{\beta}^0$ passed via the corresponding constructor arguments.

A `summary` method reports the fitted dimensions, active-coefficient count, discovered cluster count, sign-gated cell count, and mean $R^2$ in one call:

```
>>> print(model.summary())

LassoModel summary
----------------------------------------
model_type        : HIERARCHICAL_CLUSTER_GROUP_LASSO
responses (N)     : 50
factors (M)       : 9
reg_lambda        : 0.001
l1_weight (alpha) : 0
active coefs      : 346 / 450 (76.9%)
clusters (HCGL)   : 12
sign-gated cells  : 37
mean R^2          : 0.8498
```

Because `LassoModel` implements the **scikit-learn** estimator protocol (`get_params`, `set_params`, the estimator tags hook, and `fit`/`predict`/`score` accepting either **pandas** or **NumPy** inputs), it composes directly with the **scikit-learn** model-selection machinery. Regularisation strength can be selected by cross-validation without any wrapper code:

```
>>> from sklearn.model_selection import GridSearchCV
>>> search = GridSearchCV(
...     LassoModel(model_type=LassoModelType.HIERARCHICAL_CLUSTER_GROUP_LASSO),
...     param_grid={"reg_lambda": [1e-4, 1e-3, 1e-2]},
...     cv=3,
... ).fit(X, Y)
>>> search.best_params_["reg_lambda"]

0.01
```

The same protocol allows the estimator to be used inside a `sklearn.pipeline.Pipeline` (e.g., preceded by a scaler) and scored with `cross_val_score`. Each grid point and cross-validation fold is an independent convex solve, so the cost of the search is the product of the

fold count and the grid size with the per-fit time of Table 3. The package does not warm-start across $\lambda$: CVXPY rebuilds and re-solves each problem from scratch, so a dense fold-by-grid search multiplies the per-fit seconds directly. For the panel sizes of multi-asset CMA estimation the total stays in seconds, but a large grid on a wide universe is best parallelised across the `GridSearchCV n_jobs` workers. The `score` method returns the mean per-asset $R^2$, the **scikit-learn** regression-scoring convention. The fitted sign matrix can be inspected visually with `model.plot_signs()`, which renders the $\{-1, 0, +1\}$ matrix as a heatmap.

## 4. Comparison with existing packages

This section positions **factorlasso** against six existing sparse-regression packages spanning the Python and R ecosystems: **scikit-learn**'s `Lasso` class (Pedregosa *et al.* 2011), **gglasso** (Yang and Zou 2015), **sparsegl** (Liang *et al.* 2024), **asgl** (Méndez-Civieta, Aguilera-Morillo, and Lillo 2021), **celer**, and **adelie** (Yang and Hastie 2024). The comparison is on thirteen feature dimensions covering both algorithmic capabilities and the software conveniences that matter for multi-asset factor estimation in practice.

Table 2 reports the comparison. The dimensions are ordered roughly by ascending specificity to the multi-asset factor model setting. Rows 1–4 cover regularisation primitives common to most LASSO-family software. Rows 5–7 cover the HCGL cluster discovery, cluster-pooled sign derivation, and noise-floor gate that are the methodological contributions of Section 2. Row 8 covers the adaptive reweighting of Zou (2006) and Wang and Leng (2008) routed through both the $\ell_1$ and group $\ell_2$ paths. Rows 9–13 cover the software conveniences (prior centring, observation weighting, NaN tolerance, factor covariance assembly, and the reproducible simulation harness shipped with the package) that a deployed pipeline and the replication of this study require.

Several dimensions in Table 2 merit elaboration because the operational cost of working around them in a deployed pipeline is the implicit motivation for the **factorlasso** package.

**HCGL cluster discovery.** The data-driven Ward-linkage partition described in Section 2.3 is the feature that most distinguishes **factorlasso** from the other packages. Every package with group LASSO functionality (**gglasso**, **sparsegl**, **asgl**, **adelie**) requires the practitioner to supply the group structure as an external input. In multi-asset settings the appropriate group structure is rarely known a priori. Asset-class taxonomies provide a starting point but mask within-class heterogeneity and the regime-dependent restructuring of correlations under stress, when historically uncorrelated asset classes collapse into a single risk-on factor. Requiring the practitioner to choose groups upstream creates a tunable hyperparameter that the methodology of this paper eliminates: clusters are discovered from $\mathrm{corr}(\boldsymbol{Y})$, the cluster-pooled sign derivation operates on the same partition, and the row aggregation in the adaptive group penalty (Equation 11) uses the same partition again. The single-partition coherence of Section 2.3 is what gives the package its substantive coherence rather than a collection of independently-tunable components.

**Cluster-by-factor block penalty.** On the discovered partition **factorlasso** offers two group penalties that differ in which axis of the loading matrix the norm groups. The row-grouped penalty (HCGL) takes the $\ell_2$ norm over each asset's factor loadings. The cluster-by-factor

| Feature | **sklearn** | **gglasso** | **sparsegl** | **asgl** | **celer** | **adelie** | **factorlasso** |
|---|---|---|---|---|---|---|---|
| Multi-response interface ($N \times M$ in one fit) | – | – | – | – | – | ✓ | ✓ |
| $\ell_1$ penalty (LASSO) | ✓ | – | ✓ | ✓ | ✓ | – | ✓ |
| Group $\ell_2$ penalty (group LASSO) | – | ✓ | ✓ | ✓ | – | ✓ | ✓ |
| Sparse-group mixing ($\alpha \in [0, 1]$) | – | – | ✓ | ✓ | – | – | ✓ |
| HCGL cluster discovery (clusters found internally) | – | – | – | – | – | – | ✓ |
| Cluster-by-factor block penalty (FCGL) | – | – | – | – | – | – | ✓ |
| Sign constraints at cell level | – | – | – | – | – | – | ✓ |
| Noise-floor $t$-statistic gate | – | – | – | – | – | – | ✓ |
| Adaptive penalty reweighting | – | – | – | ✓ | – | – | ✓ |
| Prior-centred shrinkage ($\boldsymbol{\beta}^0 \neq \mathbf{0}$) | – | – | – | – | – | – | ✓ |
| EWMA observation weighting | – | – | – | – | – | – | ✓ |
| NaN-tolerant API | – | – | – | – | – | – | ✓ |
| Factor covariance assembly | – | – | – | – | – | – | ✓ |
| Implementation language | Python | R | R | Python | Python | Python | Python |

Table 2: Feature comparison of **factorlasso** against six sparse-regression packages in the Python and R ecosystems. A checkmark indicates that the feature is provided natively by the package. A dash indicates that the feature is absent or must be implemented externally by the practitioner. **sklearn** refers specifically to the `Lasso` class. The related `MultiTaskLasso` class supplies a different multi-response formulation that enforces shared sparsity pattern rather than the multi-response factor model fit of **factorlasso**. This feature table and the empirical benchmark of Section 5.5 cover different package sets by design. The benchmark runs **scikit-learn**, **skglm**, and **asgl** as per-asset estimators with OLS as the unregularised reference: the comparators that are installable in one Python environment and that map onto the per-asset fit the benchmark contrasts against. **skglm** is the benchmarked group-LASSO comparator and is omitted from this capability table only to keep the columns to the canonical six; **celer**, **gglasso**, and **adelie** are shown here for capability coverage but are not run, and the R-only **sparsegl** is a registered stub whose cross-language bridge is deferred to future work, so it is not part of the benchmark.

penalty (FCGL, Equation 8) takes the $\ell_2$ norm over the loadings of a cluster's assets on each factor, so the cluster is the group of the norm itself. The second grouping is intrinsically multi-response: it couples several assets inside one norm and cannot be expressed in a single-response solver. Every group-LASSO comparison package (**gglasso**, **sparsegl**, **asgl**, **adelie**) groups over the features of one response, so none can represent the cluster-by-factor block even when run per asset. The two modes are reported as alternatives in Section 5.5.

**Cell-level sign constraints with a noise-floor gate.** None of the six comparison packages implements cell-level sign constraints. The closest published mechanism is the univariate-guided sparse regression of Chatterjee *et al.* (2025), which operates on a single response and does not address the noise-floor problem at all. The closed-form SSR identity of Equation 4 that drives the gate, and the cluster aggregation of Equation 3, are both novel to **factorlasso**. These mechanisms produce the within-cluster sign coherence that an unconstrained group LASSO does not, with empirical evidence in Section 5 that the coherence rises from

approximately 0.36 to 0.81 when the mechanism is engaged on cleanly-signed cluster data.

**Prior-centred regularisation.** Shrinkage toward a non-zero prior $\boldsymbol{\beta}^0$ is absent from every comparison package: all six shrink toward zero, recovering the OLS estimate in the unregularised limit. For an unconstrained penalty this is a convenience gap rather than a capability gap: re-centring the response (fitting $\boldsymbol{Y} - \boldsymbol{X}\,\boldsymbol{\beta}^{0\top}$ with zero-centred shrinkage and adding $\boldsymbol{\beta}^0$ back) reproduces prior-centred estimation exactly. The emulation fails precisely where this package operates. Under the cell-level sign matrix, the constraint of Equation 7 becomes a per-cell shifted bound on the centred coefficients, which none of the comparison solvers accepts, so a prior cannot be combined with sign constraints from outside the optimisation problem. The remaining workaround, post-hoc add-back adjustments to the fitted loadings, such as forcing a positive credit beta for high-yield assets whose sample correlation sits below the noise floor, compromises the alpha-beta consistency that the framework is designed to enforce by construction, since the modified loadings no longer satisfy the optimisation problem of Equation 6 and the implied covariance $\hat{\boldsymbol{\beta}}\,\boldsymbol{\Sigma}_F\,\hat{\boldsymbol{\beta}}^\top + \boldsymbol{D}$ no longer corresponds to the same $\hat{\boldsymbol{\beta}}$.

**EWMA weighting, NaN tolerance, factor covariance assembly.** These software conveniences can in principle be implemented around any existing package via pre- or post-processing layers. The cost of doing so is that the pre-processing diverges between teams and between projects, and reproducibility relies on the pre-processing being described accurately in the methods section of every downstream publication. **factorlasso** embeds all three at the API boundary, producing a single artefact that handles the full pipeline from NaN-bearing return panels with heterogeneous inception dates to a serialisable factor covariance dataclass with persistent sign-matrix audit trail.

The combinatorial pattern in Table 2 is the point. No single column covers all thirteen rows. The closest in coverage are **sparsegl** (covering rows 1–4 with the inclusion of group LASSO and sparse-group mixing) and **asgl** (covering rows 1–4 and 8 with the addition of adaptive reweighting), but both require external group inputs and lack sign constraints. The combinatorial gap that spans rows 5–7 and 9–13 simultaneously is what **factorlasso** fills.

The expressiveness carries a measured runtime cost, which Table 3 quantifies across panel widths at the dimensions of the application in Section 6 ($T = 112$, $M = 9$). The per-asset competitors scale linearly in $N$, because they solve $N$ independent fits. The **factorlasso** fit grows superlinearly because **CVXPY** assembles the $N$ per-asset cone problems into one monolithic programme and the interior-point solver does not exploit their block-separable structure. Under the full production configuration it completes in under five seconds at $N = 500$. It runs well below **asgl** (the only competitor with adaptive reweighting) at the smaller panel widths (roughly a quarter of its time at $N = 50$ and $N = 100$) and is comparable to it at $N = 500$ (around 4.5 against 4.0 seconds). The FCGL block penalty matches the HCGL row penalty in wall-clock at every width (within two percent at $N = 500$): the coupling across a cluster's assets adds cone constraints of the same order as the row-grouped norm, so the loss of block-separability changes how the problem is reasoned about but not its solve time at these dimensions. The compiled **skglm** loop is faster in absolute terms by a factor of three to four at the smaller widths and a factor of fourteen at $N = 500$. For the monthly and quarterly re-estimation cycles in production, where one fit per currency frame is required, the premium buys the constraint machinery of Table 2 at interactive speed.

| Estimator | $N = 50$ | $N = 100$ | $N = 500$ |
|---|---|---|---|
| **scikit-learn** `Lasso` (per asset) | 0.01 | 0.03 | 0.13 |
| **skglm** group LASSO (per asset) | 0.03 | 0.06 | 0.31 |
| **asgl** SGL (per asset) | 0.41 | 0.80 | 3.97 |
| `FL LASSO` (joint) | 0.04 | 0.09 | 0.74 |
| `FL HCGL+SIGN+ADAPT` (joint) | 0.10 | 0.23 | 4.45 |
| `FL FCGL+SIGN+ADAPT` (joint) | 0.10 | 0.22 | 4.53 |
| `FL SGL+SIGN+ADAPT` (joint) | 0.10 | 0.23 | 4.58 |

Table 3: Per-asset competitors scale linearly in panel width, and the joint production fit grows superlinearly but stays under five seconds at $N = 500$. Median wall-clock seconds per fit over three repeats after one discarded warm-up fit (**skglm** JIT-compiles on first call), on synthetic panels with $T = 112$, $M = 9$, at $\lambda/\lambda_{\max} = 0.02$. Each entry is the total time for the full $N$-asset panel: the per-asset competitors fit the $N$ assets in an independent loop, the joint **factorlasso** rows in one solve under the production configuration of Table 1. Measured on the fourteen-core Windows 11 workstation of Appendix C; absolute values are hardware-dependent, the relative ordering and scaling are the point. Produced by `simulations/timing_scaling.py`.

# 5. Simulation study

This section reports a simulation study designed to quantify where the cluster-pooled sign-derivation mechanism of Section 2 delivers gains over the standard sparse and group LASSO baselines. The study is run on synthetic panels generated from the harness of Appendix C, with regime parameters chosen to span the operating envelope of multi-asset factor estimation and several robustness directions besides. Each estimator row corresponds to one constructor option of Section 3.1. The exhibits therefore validate the software's design choices, not a new statistical methodology.

## 5.1. Study design

The data-generating process produces synthetic panels $\boldsymbol{Y} = \boldsymbol{X}\,\boldsymbol{\beta}_{\text{true}}^{\top} + \boldsymbol{\varepsilon}$ with controllable sample length $T$, asset count $N$, factor count $M$, true cluster count $K$, sign-mix regime (clean, mixed, or idiosyncratic), sparsity level, signal-to-noise ratio (SNR), and factor and residual covariance structure. The cluster templates are built so $\boldsymbol{\beta}_{\text{true}}$ has within-cluster sign coherence. This coherence is degraded under the *mixed* regime (a subset of cells flipped) and removed entirely under the *idiosyncratic* regime (every active cell flipped with probability one-half). The full study covers nineteen regimes. The regime axes abstract the qualitative features of the fitted multi-asset model of Section 5.5: clustered loadings, within-group sign agreement, and the factor collinearity that misattributes a weakly-identified exposure. The synthetic study varies these features in isolation, while Section 5.5 draws the data-generating process from the fitted exchange-traded-fund universe directly. The grid is a core $2 \times 2 \times 2$ ablation over sparse/dense $\times$ low/high-SNR $\times$ clean/mixed plus centre points, three sample-length stress points $T \in \{60, 120, 240\}$, an $N = 200$ scaling regime, cluster-count and covariance-structure robustness regimes, crossed with five seeds, six estimators, and five $\lambda$ values, a 2,850-cell grid. The full grid is available in the per-cell replication outputs documented in Appendix C. This section summarises the findings that bear on when to use the mechanism.

Six **factorlasso** configurations form a strict ablation: `LASSO` (cellwise $\ell_1$, no groups, no signs), `GRP-ORACLE` (group LASSO with true cluster labels supplied, the unattainable best case), `GRP-HCGL` (group LASSO with HCGL-discovered clusters), `+SIGN` (HCGL plus the gated cluster-pooled sign derivation), `+SIGN+ADAPT` (plus adaptive group-$\ell_2$ reweighting), and `SGL+SIGN+ADAPT` (plus a partially active cellwise $\ell_1$ path at `l1_weight` $= 0.1$, the most fully-featured configuration). The `LASSO` and `GRP-ORACLE` rows bracket the contribution: any gain over `LASSO` is attributable to group structure, and any gain of HCGL-plus-signs over `GRP-ORACLE` to the sign mechanism rather than to cluster information per se. Each fit is scored on support recovery $F_1$, sign-agreement rate, normalised $\beta$-MSE ($\|\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}_{\text{true}}\|_F^2$ divided by $\|\boldsymbol{\beta}_{\text{true}}\|_F^2$), cluster sign coherence (the fraction of true-cluster/factor pairs whose non-zero fitted coefficients share a sign), and factor risk-premium RMSE. For each (regime, seed, estimator) triple the reported row is the $\lambda$ minimising $\beta$-MSE (oracle selection, used to isolate the estimator-level question from $\lambda$-selection, which is not this paper's contribution).

## 5.2. Headline ablation

Table 4 reports the mean of each metric across all nineteen regimes and five seeds at the oracle-selected $\lambda$. The headline pattern is in the coherence column: the three sign-derivation rows (`+SIGN`, `+SIGN+ADAPT`, `SGL+SIGN+ADAPT`) reach cluster sign coherence of approximately 0.68 (0.677 to 0.685), roughly twice the 0.29 to 0.36 of the three unconstrained rows. The mechanism delivers the within-cluster sign coherence it is designed to deliver, and the coherence separation is many standard errors wide (SE $\approx 0.014$ on a gap of 0.39). On normalised $\beta$-MSE the fully-featured `SGL+SIGN+ADAPT` edges the unconstrained baselines (0.346 versus 0.366 for `LASSO` and 0.495 for `GRP-HCGL`), but the margin over `LASSO` is within the combined standard error and should not be read as a decisive averaged improvement. The averaged margin is modest because it mixes regimes where the mechanism wins decisively with regimes where it adds bias, a cross-section the next subsection isolates.

| Estimator | Support $F_1$ | Sign rate | $\beta$-MSE | Coherence | Risk-premium RMSE |
|---|---|---|---|---|---|
| `LASSO` | 0.657 (0.002) | 0.968 (0.003) | 0.366 (0.011) | 0.358 (0.004) | 0.068 (0.004) |
| `GRP-ORACLE` | 0.591 (0.000) | 0.978 (0.002) | 0.434 (0.013) | 0.290 (0.005) | 0.069 (0.004) |
| `GRP-HCGL` | 0.591 (0.000) | 0.977 (0.002) | 0.495 (0.017) | 0.288 (0.006) | 0.071 (0.004) |
| `+SIGN` | 0.697 (0.002) | 0.906 (0.004) | 0.367 (0.005) | 0.679 (0.014) | 0.073 (0.003) |
| `+SIGN+ADAPT` | 0.695 (0.002) | 0.907 (0.004) | 0.358 (0.005) | 0.677 (0.014) | 0.072 (0.003) |
| `SGL+SIGN+ADAPT` | 0.704 (0.002) | 0.905 (0.005) | 0.346 (0.005) | 0.685 (0.014) | 0.072 (0.003) |

Table 4: Headline ablation: mean of each metric across all nineteen regimes and five seeds at oracle-selected $\lambda$, with the standard error across seeds in parentheses. Support $F_1$ is the F1 score of the non-zero-versus-zero classification of the loading cells, and sign rate is the fraction of true-non-zero cells with correctly signed estimates. Higher is better for support $F_1$, sign rate, and coherence. Lower is better for $\beta$-MSE and risk-premium RMSE.

The sign-derivation rows score lower on sign rate ($\approx 0.91$ versus 0.97 to 0.98), a shrinkage artefact rather than a failure: the sign-rate metric is conditional on a truly non-zero cell, so a true-support cell the gate pins to zero counts as a mismatch even though it is zeroed, not wrong-signed. The same gate raises support $F_1$ from 0.59 to 0.70.

## 5.3. Where the mechanism wins

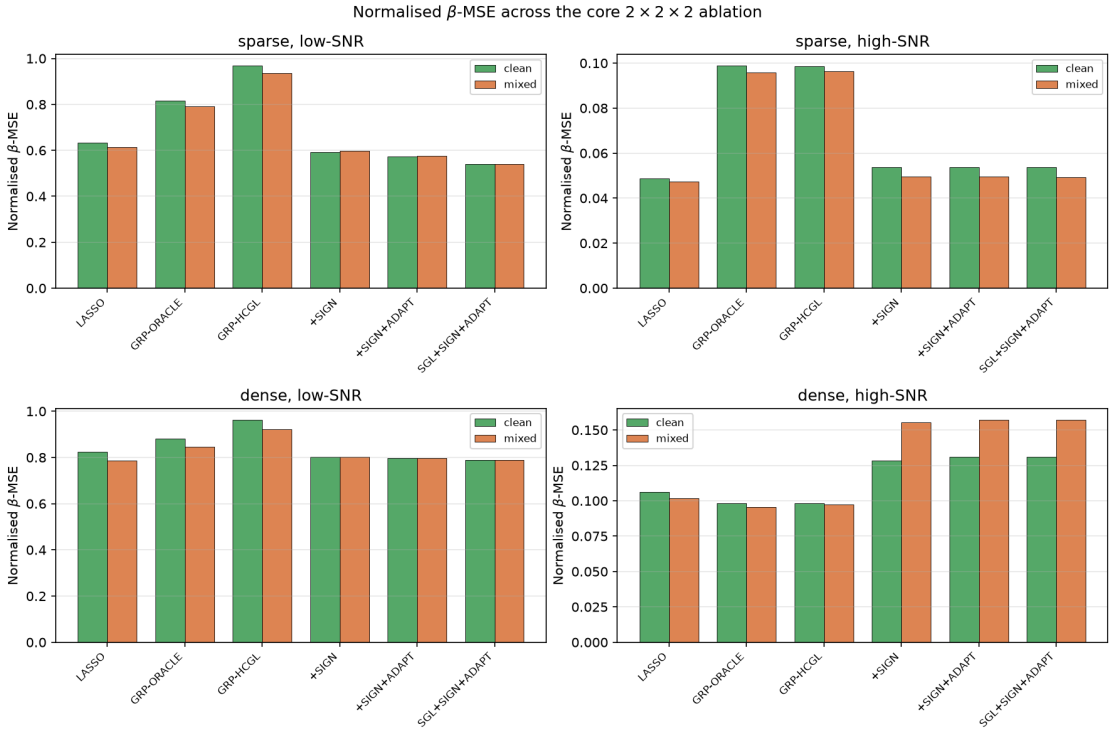


Figure 1: Normalised $\beta$-MSE across the core $2\times2\times2$ ablation, mean over five seeds at oracle-selected $\lambda$. Each panel fixes one (sparsity, SNR) combination, where SNR is the fraction of asset-return variance the factor model explains. Within each, green bars are the clean sign-mix and orange bars the mixed sign-mix (clean: within-cluster signs agree; mixed: a subset flipped). The sign-derivation estimators win most decisively in the sparse + low-SNR panel (top-left), the regime multi-asset CMA estimation occupies.

Figure 1 reports normalised $\beta$-MSE across the core $2\times2\times2$ ablation (sparse/dense $\times$ low/high-SNR $\times$ clean/mixed. The idiosyncratic regime is excluded as out of scope. The gain is regime-dependent: the sign mechanism helps most when the truth is sparse and the signal is weak, adds modest overhead when the signal is strong and sparse, and adds bias when the truth is dense and the signal strong (the standard sparsity-method failure mode). In the most favourable panel (sparse + low-SNR, clean signs) `SGL+SIGN+ADAPT` reaches $\beta$-MSE 0.54 against `GRP-HCGL`'s 0.97, a reduction of roughly forty-four percent against that baseline; in the dense + high-SNR panel the unconstrained estimators win. The regime where the mechanism operates most usefully is precisely the sparse-to-moderate, low-SNR, short-sample regime that multi-asset capital market assumption (CMA) estimation occupies. The advantage is largest at $T = 60$ and persists in attenuated form to $T = 240$, as the $T$ slices of the calibrated benchmark (Table 5) confirm.

The complementary safety property is that the mechanism does not fabricate structure absent from the data. In the idiosyncratic regime, where every active sign is independently flipped so the truth has essentially no within-cluster coherence, the sign-derivation output coherence falls to approximately 0.17 (against 0.81 in the clean regime). A practitioner applying the mechanism to data where the within-cluster sign assumption does not hold gets a near-zero

gate output and a fit that tracks the unconstrained baseline, not spurious constraints.

### 5.4. Discussion

**HCGL is a sign-coherent grouping method, not a cluster recovery method.** The adjusted Rand index between HCGL-discovered clusters and the true clusters sits at approximately 0.23 across all regimes (well above zero but well below the oracle 1.0), while within-cluster sign coherence on the HCGL partition reaches approximately 0.81 in the clean regime. The discovered clusters do not match the data-generating partition, but their members share signs on their dominant factors, which is the only property the downstream sign derivation needs. The distinction is the one drawn in the clustering-stability literature between accuracy (agreement with a ground-truth partition, which ARI measures) and replicability (agreement of the partition under resampling), the latter being the assessable criterion when no ground truth exists (von Luxburg 2010). In the deployed setting a resampling-based replicability assessment in the spirit of Sorooshyari *et al.* (2026) is the appropriate diagnostic and is left to applied work.

Density and signal-to-noise are not knowable a priori, so the operational rule is selector-based: fit with and without the constraints and compare on the practitioner's own criterion (BIC or cross-validation), and disable the constraints when the unconstrained fit wins. Two observable signals support the choice. A high gate-abstention rate indicates weak within-cluster sign evidence, where the mechanism is inert rather than harmful. Instability of the derived sign matrix under resampling the panel indicates that the pooled signs are not reproducible and should not be imposed as hard constraints.

### 5.5. Calibrated multi-asset benchmark against competing packages

The ablation above isolates the sign mechanism on synthetic templates. This subsection asks the complementary question a practitioner cares about: on a data-generating process calibrated to a real multi-asset universe, with the true loadings known, does the full **factorlasso** stack (including the economic prior) improve on the competing sparse-regression packages? We calibrate the DGP to the publicly reproducible 102-asset ETF universe of Section 6: $N = 102$ funds spanning eighteen sub-asset classes (bonds across government, investment-grade, high-yield and emerging-market issuers, regional and sector equities, hedge funds, private equity and debt, commodities, real estate, and currencies) loaded on the $M = 9$ macro factors of the MATF-CMA factor framework (equity, rates, credit, carry, inflation, commodities, private equity, rates volatility, and a zero-premium dollar factor).

Each fund's true loading vector starts from the published sub-asset-class loading profile that the fund proxies (Sepp *et al.* 2026a): each exchange-traded fund is mapped to the sub-asset class it represents (regional equity, investment-grade credit, and so on), and takes that class's representative loadings as its baseline. We perturb each nonzero loading by independent Gaussian multiplicative noise with a fifteen percent relative standard deviation, fixed per fund, which preserves the sub-class zero pattern while giving the funds in a cluster distinct loading magnitudes. The perturbation breaks the exact within-cluster identity that the synthetic templates impose, so the calibrated panel tests cluster recovery against genuinely heterogeneous members. The factor covariance is the sample covariance of the nine factor-return series over the panel window. It carries a credit–equity factor correlation of 0.84, the near-collinearity that makes the credit loading statistically fragile to identify. Idiosyncratic

variances are set per fund to reproduce that fund's empirical ordinary-least-squares $R^2$, and the true clusters are the eighteen sub-asset classes. Each replication draws a training window and an equal-length test window. The base sample is $T = 112$ months, with $T \in \{60, 240\}$ as the small- and large-sample stress points.

The roster pairs four competitors against the sign-and-prior configuration. The competitors are ordinary least squares, **scikit-learn**'s per-asset `Lasso`, **skglm**'s group LASSO, and **asgl**'s sparse group LASSO, each applied per asset on the same coefficient geometry. They run against the **factorlasso** ladder of Section 5.1 and two economically-constrained configurations the competitors cannot express: `HCGL+PRIOR`, which centres the credit loadings of the IG, HY and EM sleeves on prior values of 0.20, 0.40 and 0.30, and `HCGL+SIGN+PRIOR`, which adds the cell-level economic sign matrix. The clustered **factorlasso** rows run the *production configuration* of the MATF-CMA deployment (Table 1): `cutoff_fraction = 0.40`, gate $\tau = 1.0$, and adaptive reweighting with floor 0.5, all set a priori rather than tuned on this DGP. The production EWMA span is excluded: the DGP is stationary, so time-decayed observation weights are pure efficiency loss and would not inform the comparison. Each estimator is tuned by two $\lambda$-selectors: *oracle*, the $\lambda$ minimising $\beta$-MSE (the most generous choice for each competitor) and *BIC*, which a practitioner can actually compute. Alongside the statistical metrics of Section 5.1 we score three economic quantities on the seventeen credit-bearing (IG/HY/EM) funds: the mean recovered Credit loading (true value 0.36), the error in Credit's share of those funds' systematic variance, and the relative Frobenius error of the systematic covariance $\hat{\boldsymbol{\beta}}\boldsymbol{\Sigma}_F\hat{\boldsymbol{\beta}}^\top$.

| Estimator | $\beta$-MSE | Supp. $F_1$ | Sign agr. | OOS $R^2$ | Credit $\beta$ | Risk-sh. err | Cov err |
|---|---|---|---|---|---|---|---|
| OLS | 0.620 | 0.657 | 0.891 | 0.672 | 0.350 | 0.073 | 0.123 |
| **scikit-learn** `Lasso` | 0.216 | 0.677 | 0.544 | 0.642 | 0.009 | 0.406 | 0.166 |
| **skglm** group LASSO | 0.268 | 0.657 | **0.923** | 0.651 | 0.078 | 0.307 | 0.152 |
| **asgl** SGL | 0.242 | 0.670 | 0.906 | 0.639 | 0.064 | 0.320 | 0.169 |
| `FL LASSO` | 0.219 | 0.724 | 0.652 | 0.663 | 0.036 | 0.374 | 0.137 |
| `FL HCGL+SIGN+ADAPT` | 0.224 | 0.689 | 0.843 | 0.662 | 0.089 | 0.295 | 0.137 |
| `FL FCGL+SIGN+ADAPT` | 0.188 | 0.733 | 0.808 | 0.681 | 0.151 | 0.233 | 0.120 |
| `FL SGL+SIGN+ADAPT` | 0.218 | 0.698 | 0.834 | 0.659 | 0.088 | 0.294 | 0.141 |
| `FL HCGL+SIGN+PRIOR` | 0.185 | 0.731 | 0.848 | 0.663 | 0.319 | 0.127 | 0.140 |
| `FL FCGL+SIGN+PRIOR` | **0.161** | **0.773** | 0.811 | **0.683** | 0.309 | **0.071** | **0.119** |
| `FL ORACLE+SIGN+PRIOR` | 0.204 | 0.736 | 0.844 | 0.659 | **0.322** | 0.102 | 0.137 |

Table 5: Calibrated multi-asset benchmark at oracle-selected $\lambda$, sample $T = 112$, mean over fifteen seeds. `FL` denotes **factorlasso**. The competitor packages are applied per asset. Lower is better for $\beta$-MSE, risk-share error, and covariance error, and higher is better for support $F_1$, sign agreement (fraction of true-non-zero cells with correctly signed estimates), and OOS $R^2$. The true Credit loading on the credit-bearing funds is 0.36. Bold marks the column-best among estimators with non-degenerate fit. Standard errors across the fifteen seeds on the two discriminating columns are small: $\beta$-MSE standard errors fall in 0.004 to 0.026 and Credit $\beta$ standard errors in 0.001 to 0.009 across the rows. The credit-recovery gap between the prior configuration and the best competitor exceeds fifty standard errors of the difference. The full ladder, the BIC operating points, and the $T \in \{60, 240\}$ slices are produced by the reproduction scripts of Appendix C.

At oracle $\lambda$ the packages are statistically close (Table 5): normalised $\beta$-MSE spans 0.16 to 0.27 across the regularised estimators, and the two prior-centred configurations lead the

column, the FCGL mode at 0.161 and the HCGL mode at 0.185. **factorlasso**'s sign-constrained and adaptive configurations match or edge every competing package on support $F_1$ (0.69–0.70 versus 0.66–0.68), out-of-sample $R^2$ (0.66 versus 0.64–0.65), and systematic-covariance error (0.14 versus 0.15–0.17). The sign-derivation and adaptive layers buy their structural properties without a fit penalty, exactly as on the synthetic data. Sign agreement with the true loadings reads differently here, and the dense calibrated loadings explain why: the truth is dense, so every cell an $\ell_1$ penalty or the noise-floor gate zeroes is a true exposure counted as a sign miss. **scikit-learn**'s per-asset $\ell_1$ collapses to 0.54, the **factorlasso** LASSO sits at 0.65, and the gated configurations ($\tau = 1.0$) sit at 0.83 to 0.85 — below the ungated group estimators at 0.91 to 0.92, which zero whole groups rarely. The column orders estimators by how much they shrink or gate, not by economic fidelity: the metric artefact established on the synthetic ablation, amplified by a dense truth. Runtime favours the compiled solvers at this scale, consistent with the **CVXPY** positioning of Section 3.3 and the scaling exhibit of Table 3.

The economic columns tell a different story, and it is the central one for multi-asset CMA estimation. Against a true mean Credit loading of 0.36 on the seventeen credit-bearing funds, every penalised competitor recovers at most 0.08: under the 0.84 credit–equity collinearity the penalised estimators shrink the weakly-identified credit loading toward zero and absorb the exposure into the equity factor. **scikit-learn**'s pure $\ell_1$ zeros it outright (0.009). The adaptive reweighting recovers part of it (0.09). Only the economic prior recovers it in full (0.32), cutting the credit risk-share error to well under half of the group-LASSO competitors' (0.13 against 0.31–0.32). A prior alone is not the differentiator: re-centring the response reproduces prior-centred shrinkage around any of these packages. What no competing API expresses is the prior *jointly* with cell-level sign constraints and internally discovered groups. This integrated configuration recovers the credit loading in full while attaining a $\beta$-MSE (0.185) below every competing package. Ordinary least squares also recovers the loading (0.35), but only by declining to regularise, at the worst $\beta$-MSE in the table (0.62).

The `ORACLE+SIGN+PRIOR` row isolates the value of cluster *discovery*: it runs the identical sign, adaptive, and prior machinery but pools signs over the true sub-asset-class taxonomy rather than the HCGL-discovered partition. Discovery matches the known taxonomy on the economic quantities (credit recovery 0.319 versus 0.322, a tie within standard error) and slightly improves on $\beta$-MSE (0.185 versus 0.204), so the data-driven partition is at least as good as a hand-maintained taxonomy as the pooling unit and costs nothing when no taxonomy is available. The same holds without the prior: `HCGL+sign+adapt` and `ORACLE+sign+adapt` are within standard error on both $\beta$-MSE (0.224 versus 0.236) and support $F_1$ (0.689 versus 0.694). Discovery is the convenience the package provides, not a source of attribution error relative to the oracle grouping.

The `FCGL` rows report the cluster-by-factor penalty of Equation 8 under the otherwise identical sign-and-adaptive and sign-and-prior configurations. On this calibrated DGP, in which the true loadings share a per-sub-class centre, FCGL improves on its HCGL counterpart on every fit and economic column: paired across the fifteen seeds, $\beta$-MSE falls by 0.036 without the prior and 0.024 with it, support $F_1$ rises by about 0.043, out-of-sample $R^2$ by about 0.019, and the credit risk-share error by about 0.06, each more than two paired standard errors. The one column that moves the other way is sign agreement, which falls by about 0.035: the per-block norm zeros a whole cluster-by-factor block at once, so a block it sets to zero counts every member as a sign miss, the same metric artefact that the gated estimators show

against the dense truth. The two modes trade differently against the same metrics, and the gap reflects the match between the penalty and the data: the cluster-by-factor norm rewards a truth whose within-cluster loadings are close, while the row norm makes no homogeneity assumption. We report both rather than rank them, because the ranking is a property of the application's cluster structure rather than of the estimator. Appendix C documents a sweep over the within-cluster heterogeneity of the truth and over an alternative ground truth centred on per-asset OLS loadings, under which the ordering on support recovery reverses, confirming that neither mode dominates across data-generating assumptions.

Figure 2 contrasts the two $\lambda$-selectors and sharpens the point under a feasible criterion. Because the empirical loadings are dense, BIC (which trades fit against parameter count) finds no sparsity to reward and selects its least-regularised grid point, at which **skglm** and unconstrained **factorlasso** revert *exactly* to OLS ($\beta$-MSE 0.62). Only **factorlasso**'s sign-, adaptive-, and prior-constrained configurations reach a genuinely regularised operating point under BIC, holding $\beta$-MSE well below OLS (0.27 for the sign and adaptive configurations, 0.24 for the prior). On an economically dense truth the structure, not the $\lambda$-tuning, is what lets a feasible selector return a stable fit at all. The unstructured group methods can only revert to the high-variance least-squares estimate. The small-sample behaviour follows the same pattern across the $T$ slices of Table 5: the prior's credit-recovery advantage is widest at $T = 60$ (it recovers 0.32 against the competitors' 0.06) and narrows only modestly by $T = 240$ (where the competitors reach 0.12), while $\beta$-MSE falls for every estimator as the sample grows.

Two qualifications are owed. The prior values (0.20/0.40/0.30) sit close to the calibration truth, so the credit-recovery comparison is partly favourable by construction. But under 0.84 collinearity the credit loading is not identified from the panel alone, which is precisely why OLS is unstable and every penalised competitor zeros it, so the external prior performs necessary structural work rather than fitting in-sample noise. And the adjusted Rand index between the discovered clusters and the eighteen sub-asset labels is low (approximately 0.2), for the reason established in Section 5.4: the factor structure crosscuts the product taxonomy (rate-sensitive bonds cluster with government bonds, risk-on high-yield with equities), so the discovered partition is an economic correlation grouping, not a recovery of the sub-asset names, which is the property the downstream pooling actually requires. A prior set near the calibration truth recovering that truth is weak evidence on its own. Section 5.6 therefore stress-tests the prior away from the truth, sweeping the credit prior across a wide band and into the wrong sign, and reports how credit recovery, $\beta$-MSE, and covariance error degrade as the prior is mis-specified.

### 5.6. Prior sensitivity

The credit-recovery result of Section 5.5 uses a prior centred near the calibration truth, so it must be read against the deployment failure mode that matters: a mis-specified prior. We scale the credit-prior block of the `FL HCGL+SIGN+PRIOR` configuration by a multiplier $m$ across a wide band, holding the DGP, the fit configuration, and the sign overlay fixed, and re-run the oracle-$\lambda$ benchmark at $T = 112$. The baseline credit prior is $(\mathrm{IG}, \mathrm{HY}, \mathrm{EM}) = (0.20, 0.40, 0.30)$; $m = 1$ is the configuration of Section 5.5, $m = 0$ drops the credit prior, and $m < 0$ supplies an economically wrong-signed prior. Table 6 reports the result.

Two patterns matter. Credit recovery moves with the prior, near-linearly from 0.09 at $m = 0$ to 0.57 at $m = 2$, which confirms that the prior, not the sign or adaptive machinery, drives

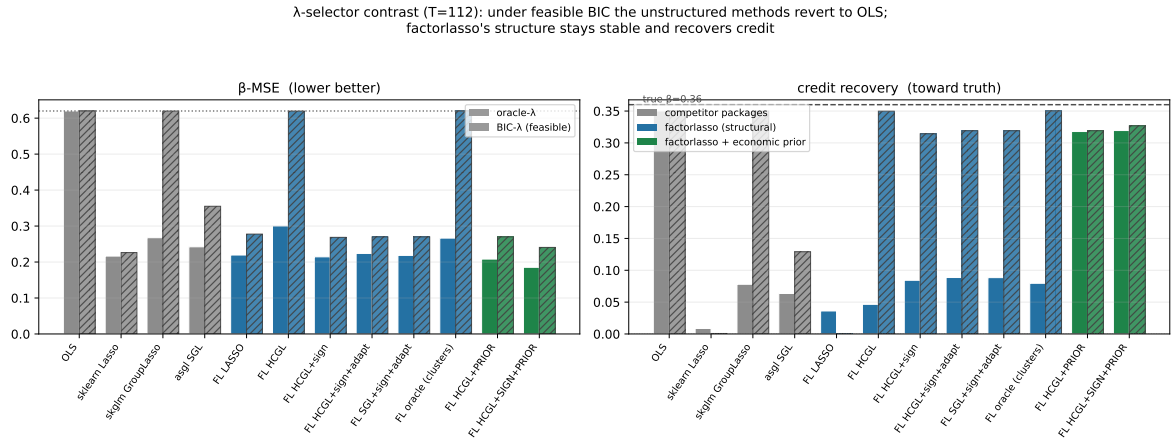


Figure 2: Calibrated multi-asset benchmark, $T = 112$: normalised $\beta$-MSE (left) and recovered Credit loading (right) by estimator under oracle-$\lambda$ (solid) and feasible BIC (hatched). The dotted line marks the OLS $\beta$-MSE and the dashed line the true Credit loading 0.36. Among the regularised estimators, only the economic prior (green) recovers the full credit loading at the $\beta$-MSE-optimal (oracle) operating point. Under BIC the unconstrained group-LASSO estimators revert to the high-variance OLS fit ($\beta$-MSE 0.62) while the sign, adaptive (blue) and prior (green) **factorlasso** configurations retain a regularised fit.

the credit-attribution result. But $\beta$-MSE stays in a narrow band (0.19 to 0.21) and covariance error is flat (0.13) across the entire sweep, including the wrong-signed priors at $m < 0$. The sign constraints and the data term bound how far a bad prior can move the fit: a wrong-signed credit prior ($m = -1$) is held near zero (0.04) rather than driven negative, because the sign constraint of Equation 7 blocks the wrong-signed pull. The prior is thus a deliberate-use tool that helps when it is correct and is bounded-harmless on $\beta$-MSE and covariance error when it is wrong, rather than a free parameter that silently determines the fit. The practitioner who supplies a prior on a well-identified loading would bias it (Section 2.6); the value of the mechanism is confined to the weakly-identified, sign-known cells for which it is designed.

## 6. Application: multi-asset capital market assumptions

The deployed setting that motivates **factorlasso**, the multi-asset allocation framework of Sepp *et al.* (2026b), is capital market assumption (CMA) estimation: the loadings of a broad cross-section of asset-class proxies on a handful of macro factors, used to build forward-looking risk and return assumptions. This section runs that setting on a publicly reproducible universe to show that the estimator behaves on market data as the calibrated simulation of Section 5.5 predicts against ground truth. This is a demonstration of estimator behaviour, not an empirical study: real loadings are unobservable, so the section does not claim that the prior-centred estimates forecast better or build better portfolios than the alternatives. The claim is the narrower one that the misattribution the simulation diagnoses under collinearity is visible on real data, and that the prior holds the credit loading where the unconstrained penalty discards it. Out-of-sample predictive and portfolio evaluation is the province of applied work using the estimator, not of this software description.

| $m$ | prior credit | credit recovery | $\beta$-MSE | cov. error |
|---|---|---|---|---|
| −1.00 | −0.29 | 0.04 | 0.21 | 0.13 |
| −0.50 | −0.15 | 0.02 | 0.21 | 0.13 |
| +0.00 | +0.00 | 0.09 | 0.20 | 0.13 |
| +0.50 | +0.15 | 0.20 | 0.19 | 0.13 |
| +0.75 | +0.22 | 0.26 | 0.19 | 0.13 |
| +1.00 | +0.29 | 0.32 | 0.19 | 0.14 |
| +1.50 | +0.44 | 0.44 | 0.19 | 0.13 |
| +2.00 | +0.59 | 0.57 | 0.20 | 0.13 |

Table 6: Prior misspecification: credit recovery tracks the prior, but $\beta$-MSE and covariance error are bounded across the full sweep, including a wrong-signed prior. Credit-prior block scaled by $m$ ($m = 1$ is the baseline configuration), `FL HCGL+SIGN+PRIOR` at oracle-$\lambda$, $T = 112$, mean over fifteen seeds. The true mean credit loading on the credit-bearing funds is 0.36. Produced by `applications/prior_sensitivity_study.py`.

### 6.1. Data

The cross-section is a curated universe of $N = 102$ exchange-traded funds spanning eighteen sub-asset classes. The fixed-income side covers government, investment-grade, high-yield and emerging-market bonds and a residual fixed-income sleeve. The equity side covers North American, European, Japanese, Asia-ex-Japan and emerging-market ex-Asia equities together with the nine US sector funds. The alternatives side covers hedge funds, private equity and private debt, precious and non-precious commodities, real estate, and seven currencies. The ticker list, sub-asset-class labels, and download script (`fetch_etf_panel.py`, built on **yfinance**, Aroussi 2025) ship with the package, so the panel regenerates without a data licence and without a holdings snapshot.

Total-return adjusted prices are resampled to month-end and converted to log returns. The thirteen-week US Treasury-bill yield is converted to a monthly log rate and subtracted to give excess returns, matching the convention of the macro-factor return series. Those nine factor series (equity, rates, credit, carry, inflation, commodities, private equity, rates volatility, and a zero-premium dollar factor) are the excess-return factor portfolios of the CMA framework (Sepp *et al.* 2026a). Aligning the two yields a balanced panel of $T = 112$ months (February 2017 through mid-2026) $\times$ $N = 102$ funds $\times$ $M = 9$ factors. Over this window the credit and equity factors carry a return correlation of 0.84 (the collinearity at the centre of the simulation), with rates-volatility/equity at $-0.42$ and inflation/commodities at 0.72. All estimates in this section use the production configuration of Table 1 (`cutoff_fraction = 0.40`, gate $\tau = 1.0$, adaptive reweighting with floor 0.5, and the sub-asset-class sign overlay shipped with the package) under uniform observation weights. The production EWMA span is a non-stationarity device and is omitted here so that the article runs one weighting convention throughout.

### 6.2. Credit attribution under shrinkage

We estimate the loadings under increasing regularisation and track the credit attribution of the seventeen IG/HY/EM bond funds, instruments that are, by construction, credit exposures. Figure 3 plots their mean Credit loading against the normalised penalty strength, for the

HCGL shrink-to-zero estimator, the prior-centred HCGL configuration, and the prior-centred FCGL configuration, with the unregularised ordinary-least-squares estimate as reference.

The ordinary-least-squares mean Credit loading is 0.82. As the penalty strengthens, the shrink-to-zero estimator drives it to zero. Under the 0.84 credit–equity collinearity the credit loading is the weakly-identified direction, so the group penalty removes it and reassigns the exposure to the equity factor (whose mean loading on these funds rises from approximately zero to 0.11 at the moderate penalty of Table 7). The prior-centred estimator instead holds the loading at its economic prior (a sleeve-weighted target of 0.29) across the entire path. The prior-centred FCGL estimator retains more credit attribution than this under shrinkage: its cluster-by-factor penalty shrinks each sleeve's deviation from the prior as a block, so the high-signal high-yield and emerging-market sleeves stay above the prior (the FCGL column of Table 7) while the low-signal investment-grade sleeve settles at it. The per-fund picture (Table 7) is the same at a moderate penalty: the Credit loading of HYG collapses from 0.85 to 0.04 while the prior holds it at 0.41. EMB collapses from 1.16 to 0.08 against a prior-held 0.35, and LQD from 0.60 to 0.04 against 0.22.

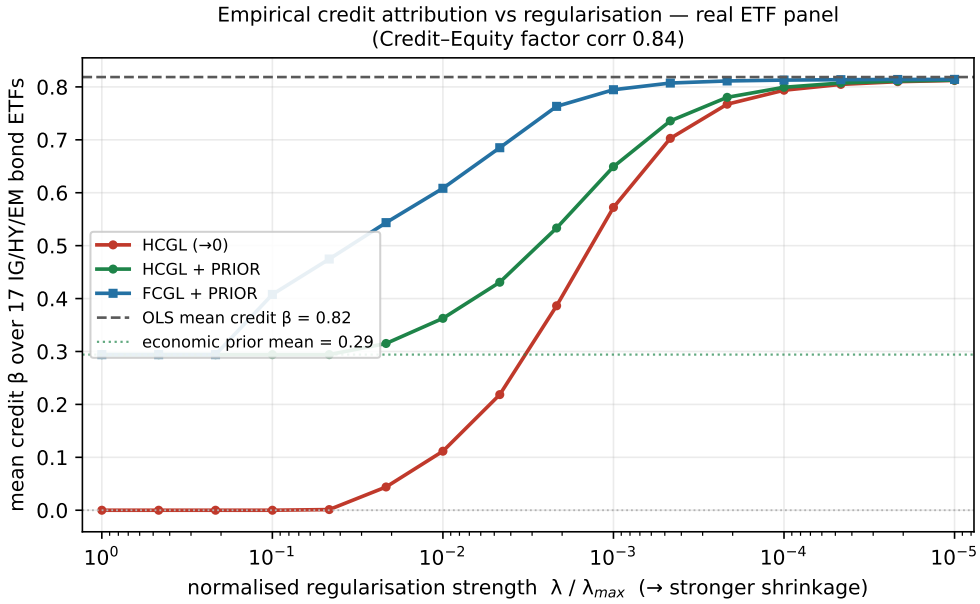


Figure 3: Mean Credit loading of the seventeen IG/HY/EM bond ETFs against normalised regularisation strength $\lambda/\lambda_{\max}$ (stronger shrinkage to the left), on the 2017–2026 excess-return panel. The shrink-to-zero HCGL estimator (red) collapses the credit attribution to zero, absorbing it into the equity factor. The prior-centred HCGL estimator (green) holds it near the economic prior. The prior-centred FCGL estimator (blue) retains more credit attribution under shrinkage, because its cluster-by-factor penalty keeps the high-signal high-yield and emerging-market sleeves above the prior rather than pulling every sleeve to it. The dashed line marks the unregularised OLS mean Credit loading, 0.82.

This is the simulation result of Section 5.5 in market data, and it carries the same reading. There is no ground-truth loading to recover here, but the behaviour is unambiguous: an unconstrained penalty asked to attribute risk across collinear macro factors zeros the credit factor on instruments that are definitionally credit and books the exposure as equity, and

| | | Credit $\beta$ | | | | Equity $\beta$ | | |
|---|---|---|---|---|---|---|---|---|
| ETF | Sleeve | OLS | shrink | prior | FCGL | OLS | shrink | prior |
| LQD | IG | 0.60 | 0.04 | 0.22 | 0.20 | 0.01 | 0.10 | 0.06 |
| USIG | IG | 0.53 | 0.02 | 0.20 | 0.20 | -0.01 | 0.05 | 0.01 |
| HYG | HY | 0.85 | 0.04 | 0.41 | 0.61 | 0.01 | 0.10 | 0.01 |
| JNK | HY | 0.90 | 0.05 | 0.41 | 0.64 | 0.01 | 0.11 | 0.03 |
| EMB | EM | 1.16 | 0.08 | 0.35 | 0.92 | -0.05 | 0.17 | 0.11 |
| PCY | EM | 1.44 | 0.14 | 0.40 | 1.13 | -0.03 | 0.27 | 0.21 |

Table 7: Per-fund Credit and Equity loadings for representative IG/HY/EM bond ETFs, at a moderate penalty ($\lambda/\lambda_{\max} = 0.02$): unregularised OLS, HCGL shrink-to-zero, the HCGL prior-centred configuration, and the FCGL prior-centred configuration (Credit $\beta$ only). Shrink-to-zero collapses the credit loading and raises the equity loading to absorb it. The HCGL prior holds the credit loading near its sleeve prior. The FCGL prior shrinks the cluster's credit loadings toward the prior as a block: the low-signal IG sleeve collapses to the prior (0.20), while the HY and EM sleeves, whose credit signal is strong, retain loadings above the prior.

no penalty strength repairs it. For a CMA engine whose downstream consumers decompose portfolio risk by factor, the economic prior is what holds the attribution in place.

## 7. Summary and discussion

The **factorlasso** package supplies a sparse multi-response factor model estimator with a cluster-pooled sign-derivation layer not available in any single existing package. The methodology of Section 2 combines four ingredients that no published software combines: hierarchical clustering group LASSO discovery of asset clusters, cluster-pooled univariate slopes filtered by a closed-form noise-floor $t$ statistic, the derived sign matrix enforced as hard convex constraints inside a CVXPY-formulated sparse group LASSO, and Zou–Wang–Leng adaptive penalty reweighting on both the $\ell_1$ and group $\ell_2$ paths. The package is deployed in a multi-asset allocation framework (Sepp *et al.* 2026b), supporting quarterly capital market assumption estimation (Sepp *et al.* 2026a).

The simulation study of Section 5 reports three empirical findings. First, the cluster-pooled sign mechanism delivers the within-cluster sign coherence it is designed to deliver: from approximately 0.36 on the unconstrained baselines to approximately 0.81 when the mechanism engages on cleanly-signed cluster data. Second, the accuracy gain is concentrated in the small-sample, low-SNR, sparse-loadings regime that multi-asset capital market assumption estimation occupies: in the most favourable panel the reduction in normalised $\boldsymbol{\beta}$-MSE against the unconstrained group LASSO is roughly forty-four percent (Figure 1), and the margin over plain LASSO is regime-specific rather than uniform. Third, the mechanism is self-aware: on the idiosyncratic regime where the within-cluster sign assumption is broken by construction, the output coherence falls to approximately 0.17 rather than a fabricated value, so the mechanism abstains when the data do not support it.

The calibrated multi-asset benchmark of Section 5.5 adds two findings the synthetic ablation cannot reach. On a data-generating process matched to the 102-fund ETF universe, with

the true loadings known, **factorlasso**'s sign-constrained and adaptive configurations match or edge **scikit-learn**, **skglm**, and **asgl** on support recovery, out-of-sample $R^2$, and systematic-covariance error, so the structural machinery costs nothing in generic accuracy. But under the 0.84 credit–equity collinearity the universe carries, every competing package and every unconstrained **factorlasso** configuration shrinks the weakly-identified credit loading toward zero and absorbs the exposure into equity. The sign- and prior-constrained configuration alone recovers it (0.32 against a true 0.36, versus at most 0.08 for the competing packages), cutting the credit risk-share error to well under half of the group-LASSO competitors'. The capability gap is the integrated estimator: a prior alone can be re-centred around any package, but not jointly with cell-level sign constraints. Under a feasible BIC the contrast sharpens: on the dense true loadings of the calibrated DGP the unconstrained group-LASSO estimators revert exactly to ordinary least squares (BIC finds no sparsity to reward) while only **factorlasso**'s sign-, adaptive-, and prior-constrained configurations reach a stably regularised fit. The same pattern appears directly in market data in the ETF application of Section 6.

**Practical recommendations.** As Section 5.4 shows, the full `SGL+SIGN+ADAPT` configuration (sparse group LASSO with sign constraints and adaptive reweighting) is the default in the financial-panel regime of short samples, sparse loadings, and low signal-to-noise ratios. The operational rule is selector-based: where density and signal strength are not known a priori, fit with and without the gate and disable it (`auto_sign_constraints = False`) when a selector prefers the unconstrained fit. The mechanism's safety property makes this low-risk, since on data without within-cluster sign coherence the derived constraints are near-trivial and the fit tracks the unconstrained baseline.

**Limitations.** Three limitations of the present work merit mention. The noise-floor threshold is held at its package default $\tau = 0.75$ across the synthetic ablation regimes of Section 5, while the calibrated benchmark and application run the stricter deployed value $\tau = 1.0$. A theoretically optimal $\tau$ would be sample-size and signal-to-noise dependent, and a sample-adaptive variant of the gate is worth exploring. The HCGL clustering recovers the true partition only weakly (Section 5.4), so replicability under resampling rather than agreement with a ground-truth partition is the criterion to assess in deployment. Finally, the economic-attribution result of Section 5.5 depends on a supplied prior: where the constrained loading is genuinely weakly identified (as the credit factor is under high credit–equity collinearity) the prior performs necessary structural work, but a prior imposed on a well-identified loading would bias it, so the mechanism is a deliberate-use tool rather than an always-on default.

**Future directions.** Several extensions are tracked in the package roadmap. The calibrated benchmark of Section 5.5 wires **scikit-learn**, **asgl**, and **skglm** into the harness as per-asset estimators. The remaining comparator, the R-only **sparsegl**, is registered as a stub and would complete the horse race of Table 2 once a cross-language bridge is added. The sign-consistency result of Appendix B specialises standard adaptive group LASSO arguments (Wang and Leng 2008) to the multi-response setting with sign constraints. Completing the distributional oracle statement (Conjecture 1) and relaxing the within-cluster sign coherence condition on HCGL recovery to a partial-coherence condition with explicit error rates are natural next steps.

## Computational details

The results in this paper were obtained using Python 3.12.7 on Windows 11 with the **factorlasso** 0.5.5 package, **CVXPY** 1.7 (Diamond and Boyd 2016) with the CLARABEL solver, **NumPy** 1.26 (Harris, Millman, van der Walt, Gommers, Virtanen, Cournapeau *et al.* 2020), **pandas** 3.0 (McKinney 2010), **SciPy** (Virtanen, Gommers, Oliphant, Haberland, Reddy, Cournapeau *et al.* 2020), **joblib** (Joblib Development Team 2025), and **matplotlib** (Hunter 2007). The results carry no platform dependence: the full simulation and application pipelines were re-run on Ubuntu Linux with **CVXPY** 1.9, **NumPy** 2.4, and **pandas** 3.0, and every reported metric agrees across the two platforms to within $10^{-11}$ — identical at the displayed precision. **factorlasso** and its dependencies are available from PyPI at https://pypi.org/project/factorlasso/. Source code, tests, simulation harness, and replication materials are at https://github.com/ArturSepp/factorlasso. The package is released under the GNU General Public License, version 3 or later.

## Acknowledgments

The authors thank colleagues at LGT Bank for feedback on the methodology and its application to portfolio optimisation and multi-asset capital market assumption estimation, and the maintainers of **CVXPY** and the broader Python scientific-computing ecosystem on which **factorlasso** is built. Any errors are the author's own.

## References

Aroussi R (2025). ***yfinance**: Download Market Data from Yahoo! Finance's API*. URL https://github.com/ranaroussi/yfinance.

Black F, Litterman R (1992). "Global Portfolio Optimization." *Financial Analysts Journal*, **48**(5), 28–43. doi:10.2469/faj.v48.n5.28.

Bondell HD, Reich BJ (2008). "Simultaneous Regression Shrinkage, Variable Selection, and Supervised Clustering of Predictors with OSCAR." *Biometrics*, **64**(1), 115–123. doi:10.1111/j.1541-0420.2007.00843.x.

Chatterjee S, Hastie T, Tibshirani R (2025). "Univariate-Guided Sparse Regression." *Harvard Data Science Review*, **7**(3). doi:10.1162/99608f92.c79ff6db.

Connor G (1995). "The Three Types of Factor Models: A Comparison of Their Explanatory Power." *Financial Analysts Journal*, **51**(3), 42–46. doi:10.2469/faj.v51.n3.1904.

de Prado ML (2016). "Building Diversified Portfolios That Outperform Out of Sample." *Journal of Portfolio Management*, **42**(4), 59–69. doi:10.3905/jpm.2016.42.4.059.

Diamond S, Boyd S (2016). "**CVXPY**: A Python-Embedded Modeling Language for Convex Optimization." *Journal of Machine Learning Research*, **17**(83), 1–5. URL https://jmlr.org/papers/v17/15-408.html.

Engle R (2002). "Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models." *Journal of Business & Economic Statistics*, **20**(3), 339–350. doi:10.1198/073500102288618487.

Fu A, Narasimhan B, Boyd S (2020). "**CVXR**: An R Package for Disciplined Convex Optimization." *Journal of Statistical Software*, **94**(14), 1–34. doi:10.18637/jss.v094.i14.

Goulart PJ, Chen Y (2024). "**Clarabel**: An Interior-Point Solver for Convex Conic Programs." *arXiv preprint*. ArXiv:2405.12762, URL https://arxiv.org/abs/2405.12762.

Grant M, Boyd S, Ye Y (2006). "Disciplined Convex Programming." In L Liberti, N Maculan (eds.), *Global Optimization: From Theory to Implementation*, pp. 155–210. Springer-Verlag, New York. doi:10.1007/0-387-30528-9_7.

Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, *et al.* (2020). "Array Programming with **NumPy**." *Nature*, **585**(7825), 357–362. doi:10.1038/s41586-020-2649-2.

Hunter JD (2007). "**matplotlib**: A 2D Graphics Environment." *Computing in Science & Engineering*, **9**(3), 90–95. doi:10.1109/MCSE.2007.55.

Joblib Development Team (2025). ***joblib***: *Running Python Functions as Pipeline Jobs*. URL https://joblib.readthedocs.io/.

Ledoit O, Wolf M (2004). "A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices." *Journal of Multivariate Analysis*, **88**(2), 365–411. doi:10.1016/S0047-259X(03)00096-4.

Liang X, Cohen A, Sólon Heinsfeld A, Pestilli F, McDonald DJ (2024). "**sparsegl**: An R Package for Estimating Sparse Group LASSO." *Journal of Statistical Software*, **110**(6), 1–23. doi:10.18637/jss.v110.i06.

McKinney W (2010). "Data Structures for Statistical Computing in Python." In *Proceedings of the 9th Python in Science Conference*, pp. 56–61. doi:10.25080/Majora-92bf1922-00a.

Méndez-Civieta A, Aguilera-Morillo MC, Lillo RE (2021). "Adaptive Sparse Group LASSO in Quantile Regression." *Advances in Data Analysis and Classification*, **15**, 547–573. doi:10.1007/s11634-020-00413-8.

Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Édouard Duchesnay (2011). "**scikit-learn**: Machine Learning in Python." *Journal of Machine Learning Research*, **12**, 2825–2830. URL https://jmlr.org/papers/v12/pedregosa11a.html.

Richland J, Kiiskinen T, Wang W, Lu S, Narasimhan B, Hastie T, Rivas M, Tibshirani R (2025). "Univariate-Guided Sparse Regression for Biobank-Scale High-Dimensional Omics Data." *arXiv preprint*. ArXiv:2511.22049, URL https://arxiv.org/abs/2511.22049.

Rosenberg B, McKibben W (1973). "The Prediction of Systematic and Specific Risk in Common Stocks." *Journal of Financial and Quantitative Analysis*, **8**(2), 317–333. doi:10.2307/2330027.

Sepp A (2024). "**OptimalPortfolios**: Implementation of Optimisation Analytics for Constructing and Backtesting Optimal Portfolios in Python." URL https://github.com/ArturSepp/OptimalPortfolios.

Sepp A, Hansen E, Kastenholz M (2026a). "Capital Market Assumptions Using Multi-Asset Tradable Factors: The MATF-CMA Framework." Working paper, under review at the *Journal of Portfolio Management*.

Sepp A, Ossa I, Kastenholz M (2026b). "Robust Optimization of Strategic and Tactical Asset Allocation for Multi-Asset Portfolios." *Journal of Portfolio Management*, **52**(4), 86–120. URL https://eprints.pm-research.com/17511/143431/index.html.

Simon N, Friedman J, Hastie T, Tibshirani R (2013). "A Sparse-Group LASSO." *Journal of Computational and Graphical Statistics*, **22**(2), 231–245. doi:10.1080/10618600.2012.681250.

Sorooshyari SK, Rivas MA, Tibshirani R (2026). "ERICA: Quantifying Replicability of Cluster Analysis." arXiv:2606.00302. doi:10.48550/arXiv.2606.00302. Preprint.

Tibshirani R (1996). "Regression Shrinkage and Selection via the LASSO." *Journal of the Royal Statistical Society B*, **58**(1), 267–288. doi:10.1111/j.2517-6161.1996.tb02080.x.

Vattikuti S, Lee JJ, Chang CC, Hsu SDH, Chow CC (2014). "Applying Compressed Sensing to Genome-Wide Association Studies." *GigaScience*, **3**(1), 10. doi:10.1186/2047-217X-3-10.

Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, *et al.* (2020). "**SciPy** 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods*, **17**(3), 261–272. doi:10.1038/s41592-019-0686-2.

von Luxburg U (2010). "Clustering Stability: An Overview." *Foundations and Trends in Machine Learning*, **2**(3), 235–274. doi:10.1561/2200000008.

Wang H, Leng C (2008). "A Note on Adaptive Group LASSO." *Computational Statistics & Data Analysis*, **52**(12), 5277–5286. doi:10.1016/j.csda.2008.05.006.

Yang J, Hastie T (2024). "A Fast and Scalable Pathwise-Solver for Group LASSO and Elastic Net Penalized Regression via Block-Coordinate Descent." *arXiv preprint*. ArXiv:2405.08631, URL https://arxiv.org/abs/2405.08631.

Yang Y, Zou H (2015). "A Fast Unified Algorithm for Solving Group-LASSO Penalized Learning Problems." *Statistics and Computing*, **25**(6), 1129–1141. doi:10.1007/s11222-014-9498-5.

Yuan M, Lin Y (2006). "Model Selection and Estimation in Regression with Grouped Variables." *Journal of the Royal Statistical Society B*, **68**(1), 49–67. doi:10.1111/j.1467-9868.2005.00532.x.

Zakamulin V (2015). "A Test of Covariance-Matrix Forecasting Methods." *The Journal of Portfolio Management*, **41**(3), 97–108. doi:10.3905/jpm.2015.41.3.097.

Zou H (2006). "The Adaptive LASSO and Its Oracle Properties." *Journal of the American Statistical Association*, **101**(476), 1418–1429. doi:10.1198/016214506000000735.

# A. The closed-form SSR identity

This appendix derives the closed-form residual sum-of-squares identity of Equation 4, gives the full $t$ statistic, and states the missing-observation accounting that the estimator uses on panels with heterogeneous inception dates.

**Derivation.** For factor $j$, the pooled no-intercept slope $\hat{\beta}_j$ of Equation 2 satisfies the first-order condition $\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}} = N\,\hat{\beta}_j\,(\boldsymbol{x}_j^\top \boldsymbol{x}_j)$. The pooled residual sum of squares aggregates the squared residuals of the $N$ per-asset fits at the common slope $\hat{\beta}_j$,

$$\begin{aligned}\text{SSR}_j &= \sum_{k=1}^{N} \|\boldsymbol{y}_k - \hat{\beta}_j\,\boldsymbol{x}_j\|_2^2 = \sum_{k=1}^{N} \left(\|\boldsymbol{y}_k\|_2^2 - 2\,\hat{\beta}_j\,\boldsymbol{x}_j^\top \boldsymbol{y}_k + \hat{\beta}_j^{\,2}\,\boldsymbol{x}_j^\top \boldsymbol{x}_j\right) \\ &= \|\boldsymbol{Y}\|_F^2 - 2\,\hat{\beta}_j\,\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}} + N\,\hat{\beta}_j^{\,2}\,\boldsymbol{x}_j^\top \boldsymbol{x}_j, \end{aligned} \tag{14}$$

using $\sum_k \|\boldsymbol{y}_k\|_2^2 = \|\boldsymbol{Y}\|_F^2$ and $\sum_k \boldsymbol{x}_j^\top \boldsymbol{y}_k = \boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}}$. Substituting the first-order condition $\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}} = N\,\hat{\beta}_j(\boldsymbol{x}_j^\top \boldsymbol{x}_j)$ into the middle term collapses Equation 14 to Equation 4, $\text{SSR}_j = \|\boldsymbol{Y}\|_F^2 - N\,\hat{\beta}_j^2(\boldsymbol{x}_j^\top \boldsymbol{x}_j)$. The right-hand side requires only $\|\boldsymbol{Y}\|_F^2$ (computed once), the per-factor inner product $\boldsymbol{x}_j^\top \boldsymbol{x}_j$, and the slope, so the full $(N, M)$ statistic is one matrix product rather than a loop that materialises the $T \times N$ residual matrix per factor.

**Variance, standard error, and $t$ statistic.** The pooled variance estimator, standard error, and $t$ statistic are

$$\hat{\sigma}_j^{\,2} = \frac{\text{SSR}_j}{N\,T - N}, \qquad \widehat{\text{SE}}(\hat{\beta}_j) = \sqrt{\frac{\hat{\sigma}_j^{\,2}}{N\,\boldsymbol{x}_j^\top \boldsymbol{x}_j}}, \qquad t_j = \frac{\hat{\beta}_j}{\widehat{\text{SE}}(\hat{\beta}_j)}. \tag{15}$$

The cluster-pooled variant $t_{j,\mathcal{C}}$ used in the gate of Equation 5 substitutes $|\mathcal{C}|$ for $N$ throughout Equation 15 and restricts the Frobenius norm to the cluster columns, $\sum_{k\in\mathcal{C}} \|\boldsymbol{y}_k\|_2^2$.

**Missing observations.** Asset panels typically carry leading-`NaN` prefixes from heterogeneous inception dates and occasional mid-stream `NaN` from data-quality flags. The estimator records the per-cell validity masks on entry and zero-fills for the arithmetic: $v_{tk} \in \{0, 1\}$ flags an observed $y_{tk}$ and $u_{tj} \in \{0, 1\}$ flags an observed $x_{tj}$. A zeroed entry contributes nothing to the slope numerator $\boldsymbol{x}_j^\top \boldsymbol{y}_{\text{sum}}$ or to $\|\boldsymbol{Y}\|_F^2$. The slope denominator, the SSR, and the degrees of freedom are accumulated only over valid $(t, k)$ cells of factor $j$: the pooled denominator is $\sum_k \sum_t v_{tk}\,u_{tj}\,x_{tj}^2$ rather than $N\,\boldsymbol{x}_j^\top \boldsymbol{x}_j$, and the degrees of freedom use the valid-observation count $n_{\text{valid},j} = \sum_k \sum_t v_{tk}\,u_{tj}$ in place of $N\,T$. With this accounting the slope and $t$ statistic equal those of the no-intercept OLS fit restricted to the valid rows, and reduce exactly to Equation 15 when the panel is fully observed. The degrees-of-freedom convention $n_{\text{valid},j} - q_{\text{eff}}$ charges one parameter per pooled response column that contributes at least one valid observation ($q_{\text{eff}} = N$ in the fully observed case, giving $N\,T - N$). Alternative conventions shift $|t|$ by $O(1/T)$ and do not alter gate decisions at the default $\tau$.

# B. Theoretical detail

This appendix states the support-sign consistency property of the cluster-pooled sign-derivation mechanism and gives a proof sketch. The sign matrix is derived identically under HCGL and FCGL, which gate the same cluster-pooled slope at the same threshold and differ only in the group norm of the penalty, so the argument applies to both modes. The argument specialises the adaptive group LASSO oracle results of Wang and Leng (2008) to the multi-response setting with hard sign constraints derived from the cluster-pooled univariate slope. The sign-consistency statement on the true support (Proposition 1) is established at the level of rigour of a software-paper appendix. The oracle distributional statement that would accompany a full theorem (asymptotic normality on the true support with OLS-equivalent covariance) is recorded as Conjecture 1 and discussed but not proved here. The appendix is a heuristic justification under stylised conditions, not a claimed guarantee for the deployment regime: condition (A6) below requires the marginal cluster-pooled slope to agree in sign with the partial true loading, and this is exactly what high cross-factor collinearity breaks. Under the 0.84 credit–equity collinearity of the application the asymptotic argument is therefore not claimed to hold, and the evidence for the mechanism in that regime is the simulation of Section 5 and the prior-sensitivity study of Section 5.6, not Proposition 1.

Throughout this appendix we adopt the notation of Section 2. Let $\boldsymbol{\beta}_{\text{true}} \in \mathbb{R}^{N \times M}$ denote the true loading matrix in the data-generating model $\boldsymbol{Y} = \boldsymbol{X}\,\boldsymbol{\beta}_{\text{true}}^{\top} + \boldsymbol{\varepsilon}$, and let $\boldsymbol{S}_{\text{true}} \in \{-1, 0, +1\}^{N \times M}$ be the matrix of true signs, with $S_{\text{true},kj} = 0$ where $\beta_{\text{true},kj} = 0$. Let $\hat{\boldsymbol{S}}_T$ denote the derived sign matrix produced by the noise-floor-gated cluster-pooled mechanism of Equations 3–5 at sample size $T$, and let $\hat{\boldsymbol{\beta}}_T$ denote the `SGL+SIGN+ADAPT` estimator of Equation 6 constrained by $\hat{\boldsymbol{S}}_T$ and reweighted by Equations 9–12.

**Regularity conditions.**

**(A1) Sub-Gaussian noise.** The residual matrix $\boldsymbol{\varepsilon}$ has independent rows with mean zero and finite variance, and each entry $\varepsilon_{tk}$ is sub-Gaussian with proxy variance bounded above by a fixed constant $\sigma^2_{\max}$.

**(A2) Factor regularity.** The factor design matrix $\boldsymbol{X}$ satisfies the usual moment conditions $T^{-1}\,\boldsymbol{X}^{\top}\boldsymbol{X} \to \boldsymbol{\Sigma}_F$ as $T \to \infty$ with $\boldsymbol{\Sigma}_F$ positive definite, and the columns of $\boldsymbol{X}$ are bounded in probability.

**(A3) Cluster sign coherence.** The HCGL partition recovered from $\operatorname{corr}(\boldsymbol{Y})$ is consistent with the true within-cluster sign structure in the following sense: for every $(j, \hat{\mathcal{C}})$ pair where $\hat{\mathcal{C}}$ is a cluster discovered by HCGL on the sample $(\boldsymbol{X}, \boldsymbol{Y})$, the signs $\{S_{\text{true},kj} : k \in \hat{\mathcal{C}}, S_{\text{true},kj} \neq 0\}$ share a single value. This condition is weaker than perfect cluster recovery: the discovered partition does not need to match the true cluster identity, only to respect the true sign structure.

**(A4) Separation.** There exists a positive constant $c^{\star}$ such that, for every cluster $\mathcal{C}$ and every factor $j$ with $\beta_{\text{true},kj} \neq 0$ for some $k \in \mathcal{C}$, the cluster-pooled population slope $\beta^{\text{pop}}_{j,\mathcal{C}}$ satisfies $|\beta^{\text{pop}}_{j,\mathcal{C}}| / \operatorname{SE}_T(\beta^{\text{pop}}_{j,\mathcal{C}}) \to \infty$ at rate at least $\sqrt{T}$ as $T \to \infty$, with margin $|\beta^{\text{pop}}_{j,\mathcal{C}}| / \operatorname{SE}_T \geq \tau + c^{\star}$ for all sufficiently large $T$.

**(A5) Penalty rate.** The regularisation strength satisfies $\lambda_T \to 0$ and $\sqrt{T}\,\lambda_T \to \infty$ as $T \to \infty$. The adaptive exponent is $\gamma > 0$.

**(A6) Marginal sign agreement on the support.** For every discovered cluster $\hat{\mathcal{C}}$ and every factor $j$ with $S_{\text{true},kj} \neq 0$ for some $k \in \hat{\mathcal{C}}$, the population cluster-pooled slope satisfies $\operatorname{sign}(\beta^{\text{pop}}_{j,\hat{\mathcal{C}}}) = S_{\text{true},kj}$ for every such $k$. The condition is substantive because the pooled slope is a *marginal* quantity: with correlated factors, $\beta^{\text{pop}}_{j,\mathcal{C}} \propto \sum_l \big(\sum_{k\in\mathcal{C}} \beta_{\text{true},kl}\big)\boldsymbol{\Sigma}_{F,jl}$, so cross-factor leakage can flip the marginal sign away from the *partial* sign that $\boldsymbol{S}_{\text{true}}$ encodes. (A6) holds when the factors are uncorrelated, and more generally when the cross-factor leakage term does not dominate the own-factor term. The mixed and idiosyncratic sign-mix regimes of Section 5 probe exactly the violations of (A3) and (A6).

**Statement.** Define the support-sign event

$$\mathcal{H}_T \;=\; \Big\{ \hat{S}_{T,kj} = S_{\text{true},kj} \;\text{ for all } (k,j) \text{ with } \; S_{\text{true},kj} \neq 0 \Big\}, \tag{16}$$

the event that the gate neither pins a true-support cell to zero nor assigns it the wrong half-space.

**Proposition 1 (support-sign and support consistency).** Under conditions (A1)–(A6), $\Pr(\mathcal{H}_T) \to 1$ as $T \to \infty$, and, conditional on this event, the constrained adaptive sparse group LASSO estimator $\hat{\boldsymbol{\beta}}_T$ recovers the true support: $\Pr(\operatorname{supp}(\hat{\boldsymbol{\beta}}_T) = \operatorname{supp}(\boldsymbol{\beta}_{\text{true}})) \to 1$.

**Remark (truly-zero cells and the fixed threshold).** Proposition 1 deliberately does not claim $\Pr(\hat{\boldsymbol{S}}_T = \boldsymbol{S}_{\text{true}}) \to 1$. At a fixed threshold $\tau$, that stronger claim is false. On a cell where the population pooled slope is exactly zero, the gate $t$ statistic is asymptotically standard normal, so the gate retains the cell with limiting probability $2\Phi(-\tau) \approx 0.45$ at the default $\tau = 0.75$. On a cell where the partial coefficient is zero but the marginal pooled slope is not (cross-factor leakage), the gate retains the cell with probability tending to one. Both outcomes leave support recovery to the penalty. The retained constraint is a one-sided half-space whose boundary contains zero, so it never excludes the value $\hat{\beta}_{kj} = 0$ that the adaptive penalty drives the cell toward. A binding half-space on a truly-zero cell clips the unconstrained estimate's sampling noise on one side, a bounded bias of order the cell's standard error rather than a sign error. Exact recovery of $\boldsymbol{S}_{\text{true}}$ would additionally require a diverging threshold, $\tau_T \to \infty$ with $\tau_T = o(\sqrt{T})$, together with zero marginal leakage on the off-support cells. The implementation keeps the fixed finite-sample default and treats the gate as a screen, not as a model-selection oracle.

**Conjecture 1 (oracle distribution).** Under the same conditions, the restriction of $\sqrt{T}\,(\hat{\boldsymbol{\beta}}_T - \boldsymbol{\beta}_{\text{true}})$ to the true support $\operatorname{supp}(\boldsymbol{\beta}_{\text{true}})$ converges in distribution to a centred Gaussian with the same asymptotic covariance as the OLS estimator restricted to the true support. Establishing this requires verifying that the row-aggregated adaptive weights of Equation 11 inherit the divergence-on-zeros and boundedness-on-support rates of the Wang and Leng (2008) argument in the multi-response, sign-constrained setting. We do not prove it here.

**Proof sketch.** The argument proceeds in four steps that mirror the layered architecture of the mechanism. Steps 1–3 establish $\Pr(\mathcal{H}_T) \to 1$. Step 4 sketches the conditional support recovery and isolates what remains open for the distributional Conjecture 1.

*Step 1 (slope consistency).* Under conditions (A1)–(A2), the cluster-pooled univariate slope $\hat{\beta}_{j,\mathcal{C}}$ of Equation 3 satisfies $\hat{\beta}_{j,\mathcal{C}} \to \beta^{\text{pop}}_{j,\mathcal{C}}$ in probability as $T \to \infty$ for every cluster $\mathcal{C}$ and factor $j$. This convergence is a direct application of the law of large numbers to the no-intercept OLS estimator, using the sub-Gaussian tail bound on $\boldsymbol{\varepsilon}$ to control the deviation rate.

*Step 2 (no gating on the support).* The closed-form SSR identity of Equation 4 produces a consistent variance estimator $\hat{\sigma}^2_j \to \sigma^2_{j,\text{pop}}$ in probability. For every $(j, \mathcal{C})$ pair on the support, the separation condition (A4) then gives $|t_{j,\mathcal{C}}| \to \infty$ in probability at rate $\sqrt{T}$, so $\Pr(|t_{j,\mathcal{C}}| \geq \tau) \to 1$ and the probability that the gate pins any true-support cell to zero vanishes. The step makes no claim about off-support cells. At a fixed $\tau$ the gate retains an off-support cell with non-vanishing probability (see the Remark above), and the argument does not need it to abstain.

*Step 3 (correct signs on the support).* Steps 1 and 2 imply that for every cluster $\hat{\mathcal{C}}$ discovered by HCGL and every support factor $j$, the derived sign $\text{sign}(\hat{\beta}_{j,\hat{\mathcal{C}}})$ converges in probability to $\text{sign}(\beta^{\text{pop}}_{j,\hat{\mathcal{C}}})$. The marginal sign agreement condition (A6) equates this population sign with the common true entry $S_{\text{true},kj}$, which is single-valued within the cluster by the coherence condition (A3). The broadcasted sign therefore matches $S_{\text{true},kj}$ for every $k \in \hat{\mathcal{C}}$ on the support, and $\Pr(\mathcal{H}_T) \to 1$ follows from a union bound over the finitely many $(j, \mathcal{C})$ pairs.

*Step 4 (constrained adaptive support recovery).* Condition on the event $\mathcal{H}_T$, which has probability tending to one by Step 3. On this event, every true-support cell carries its correct half-space constraint, and the support-restricted estimator is consistent by standard arguments, so the constraint is inactive in the limit and the estimator coincides with the unconstrained adaptive group LASSO on the support. Off-support cells carry either a zero-pin (which enforces the truth directly) or a one-sided half-space containing zero (which cannot exclude the target value and can bind only by pushing the estimate toward zero). Under condition (A5), the adaptive reweighting of Equation 9 with $\gamma > 0$ produces weights that diverge on truly-zero cells and remain bounded on truly-nonzero cells, which yields support recovery by the standard adaptive-penalty argument. This establishes the support-recovery claim of Proposition 1. The further distributional claim of Conjecture 1 would require verifying that the row-aggregated adaptive weights of Equation 11 (an RMS aggregation across the non-pinned factors of an asset, not a single per-coefficient weight as in the original Wang and Leng (2008) setup) inherit the rate conditions their central-limit argument needs in the multi-response, sign-constrained problem. We leave this to future work.

The first three steps together establish the support-sign consistency and, with Step 4, the support-recovery statement of Proposition 1.

**Remarks.** The cluster sign coherence condition (A3) and the marginal sign agreement condition (A6) are the substantive sufficient conditions for the proof. (A3) is weaker than the perfect-cluster-recovery condition that would require $\hat{\mathcal{C}}_g = \mathcal{C}^\star_g$ for every cluster. It permits HCGL to discover a partition that differs from the true clusters at the identity level, as long as the discovered clusters respect the within-cluster sign structure. (A6) is the price of deriving signs from marginal rather than partial evidence, and it is the condition a practitioner should question first on universes with strong cross-factor correlation of opposing signs. The simulation evidence of Section 5.4 shows that HCGL satisfies the coherence condition reliably even when the adjusted Rand index against the true clusters is moderate.

# C. Reproducing the simulation study

This appendix gives step-by-step instructions for reproducing the full study from a fresh Python environment. The replication materials ship with the **factorlasso** source distribution under the `papers/jss_2026/simulations/` directory.

**Single replication script.** The recommended entry point is the standalone script `replicate.py` in `papers/jss_2026/`, which runs the simulation study of Section 5 (the synthetic regimes, the core-grid figure, and Table 4) in sequence and writes a captured log with a session-information footer:

```
python papers/jss_2026/replicate.py
python papers/jss_2026/replicate.py --full
```

The default quick mode runs one seed of the simulation grid and reduced seed counts for the calibrated benchmark, completing within a few minutes on a regular workstation. Quick mode reproduces the deterministic outputs exactly: the usage-example outputs of Section 3.5, the empirical application table (Table 7), and the credit-path figure. The Monte Carlo outputs (Table 4, Table 5, Figure 1, and Figure 2) emerge in shape and qualitative pattern with wider Monte Carlo error. The `-full` mode runs the full seed counts and reproduces every reported number exactly. The remaining paragraphs document the individual stages for readers who wish to run them separately.

**Software requirements.** The replication requires Python 3.12 or higher with the **factorlasso** package installed in its `simulations` optional dependency group:

```
git clone https://github.com/ArturSepp/factorlasso.git
cd factorlasso
pip install -e ".[simulations]"
```

The `[simulations]` extra brings in **PyYAML** for study configuration, **joblib** for parallel cell execution, **pyarrow** for parquet output, and **SCS** and **ECOS** as solver fallbacks for the rare problem instances where CLARABEL encounters numerical pathologies.

**Repository layout.** All artefacts tied to this paper live under `papers/jss_2026/`, organised in three subdirectories: `paper/` (LaTeX source, references, and figures), `simulations/` (the synthetic harness and the calibrated-benchmark results of Section 5.5), and `applications/` (the multi-asset ETF study of Section 6). The multi-asset benchmark is reproduced by three scripts in `applications/`: `fetch_etf_panel.py` builds the panel from the public ticker list `etf_universe.csv`, `etf_simulation_study.py` runs the calibrated study, and `plot_competitor_study.py` draws its figures. The empirical credit attribution of Section 6.2 is produced by `run_etf_study.py`. The HCGL-versus-FCGL comparison of Section 5.5 is produced by two further scripts in `applications/`: `compare_modes_on_tables.py` runs the two penalty modes through the calibrated benchmark on the same metrics as Table 5, and `recalibrate_dgp_sweep.py` sweeps the within-cluster heterogeneity of the ground truth and

the choice of per-sub-class centre. The simulation harness is not part of the published **factorlasso** wheel. It ships with the source repository, and its dependencies are installed by adding the `simulations` optional dependency group to a source checkout:

```
pip install factorlasso[simulations]
```

The harness consists of four reusable modules under the `papers/jss_2026/simulations/` directory:

- `dgp` contains the data-generating processes. A `DGPConfig` dataclass encodes the regime axes ($T$, $N$, $M$, $K$ true clusters, within-cluster $\beta$-correlation, sign mix, sparsity level, signal-to-noise ratio, factor covariance structure, residual covariance structure), and `make_synthetic_panel` produces a fully labelled **pandas** panel along with the ground truth $\beta$ and cluster assignment for any seed.
- `estimators` exposes a unified `fit(X, y, reg_lambda) -> EstimatorResult` interface and registers each of the **factorlasso** configurations of the Section 5 ablation, along with stubs for external competitors (**scikit-learn**'s `Lasso`, **sparsegl**, **adelie**) that can be wired in when their dependencies are installed.
- `metrics` contains pure functions on $(\beta_{\text{true}}, \hat{\beta}, \ldots)$ that compute the metrics used in Section 5: three mandatory metrics (support recovery $F_1$, sign agreement rate, normalised $\beta$-MSE), two secondary metrics tabulated in the headline ablation (cluster sign coherence and factor risk-premium RMSE), and a cluster-recovery adjusted Rand index (ARI) used in the discussion of Section 5.4.
- `run` is the orchestration entry point. It reads a study configuration YAML, expands the regime grid, runs the Cartesian product of cells in parallel via **joblib** (Joblib Development Team 2025), applies oracle-$\lambda$ selection over the `lambda_grid`, and writes three artefacts to disk: `results_long.parquet` carrying every fit's metrics and runtime, `results_oracle_lambda.parquet` carrying only the $\lambda$-minimising row per (regime, seed, estimator), and `manifest.json` carrying the reproducibility receipt (package version, Python version, platform, wall-clock, ok/failed/not-implemented cell counts, UTC timestamp).

The JSS study configuration is at `simulations/study.yaml`, with results written to `simulations/results/` after a run. The analysis script that produces the figures and tables of Section 5 lives at `paper/analysis.py`.

**Penalty-mode comparison and anchor sweep.** The claim that neither penalty mode dominates across data-generating assumptions is reproduced by `recalibrate_dgp_sweep.py`. The script builds the ground-truth loadings from a per-sub-class centre plus a tunable within-cluster scatter, so a scatter of zero gives a block-constant truth and a larger scatter gives a heterogeneous one. It runs HCGL, FCGL, and the per-asset LASSO baseline at each scatter level and reports the same identification metrics as Table 5. The centre is either the per-sub-class production loadings (`-anchor hcgl`) or per-asset OLS loadings (`-anchor ols`):

```
python papers/jss_2026/applications/recalibrate_dgp_sweep.py \
--anchor ols --sigmas 0.0 0.15 0.30 0.45 \
--seeds 15 --seed-start 101
```

Under the OLS-centred truth the ordering on support recovery reverses relative to Table 5, confirming that the mode ranking reflects the match between the penalty and the within-cluster structure of the application rather than a property of the estimator.

**Dry run and smoke test.** Before executing the full study grid, the dry-run mode verifies the configuration is loadable and reports the total cell count without executing any fits:

```
python -m papers.jss_2026.simulations.run \
--config papers/jss_2026/simulations/study.yaml \
--output /tmp/dry_run \
--dry-run
```

A single-seed smoke test exercises the full pipeline (data generation, estimator fit, metric computation, output write) on a fraction of the grid:

```
python -m papers.jss_2026.simulations.run \
--config papers/jss_2026/simulations/study.yaml \
--output /tmp/smoke \
--seeds-limit 1
```

The smoke test completes in approximately two minutes of wall-clock on a four-core workstation and exercises every estimator on every regime at every regularisation grid point, producing 570 rows of output.

**Full study run.** The full study grid is invoked with default parameters. The runner uses all available cores via **joblib**:

```
python -m papers.jss_2026.simulations.run \
--config papers/jss_2026/simulations/study.yaml \
--output papers/jss_2026/simulations/results
```

On a fourteen-core workstation running Python 3.12 on Windows 11, the full 2,850-cell grid completes in approximately 45 seconds of wall-clock. On a single core the same grid completes in under ten minutes. The expected output is three files in the `-output` directory: `results_long.parquet` (2,850 rows of every fit's metrics, runtime, and regime parameters), `results_oracle_lambda.parquet` (570 rows giving the $\beta$-MSE-minimising $\lambda$ per (regime, seed, estimator) triple), and `manifest.json` (the reproducibility receipt with software versions, platform, timestamp, and cell counts).

**Producing the tables and figures.** With the parquet outputs in place, the analysis script reproduces the four figures and the headline ablation table of Section 5:

```
python papers/jss_2026/paper/analysis.py \
--results papers/jss_2026/simulations/results \
--output papers/jss_2026/paper/figures
```

The script reads `results_oracle_lambda.parquet`, computes the aggregations summarised in Table 4 and the four figures referenced by Section 5, and writes the figures into the supplied output directory. The timing exhibit of Table 3 is produced separately by `simulations/timing_scaling.py`. Its absolute values are hardware-dependent by nature, so the reproduction target is the relative ordering and the scaling pattern, not the seconds.

**Auditing a single fit.** The `results_long.parquet` carries every cell's diagnostic information including the solver actually used by CVXPY for that fit. To identify cells where the solver fallback chain engaged:

```
>>> import pandas as pd
>>> df = pd.read_parquet(
...     "papers/jss_2026/simulations/results/results_long.parquet"
... )
>>> df[df["solver_used"] != "CLARABEL"][
...     ["regime_id", "seed", "estimator", "reg_lambda"] ]
```

Cells where CLARABEL succeeded report `solver_used = "CLARABEL"`. Cells where the fallback engaged (rare in practice, observed in approximately one cell per several thousand on the orthogonal factor regime) report `"ECOS"` or `"SCS"` and can be re-fit deterministically with the recorded `seed` and `reg_lambda` to verify reproducibility of the result.

**Manifest schema.** The `manifest.json` written by every run records the study name and the full config (the study YAML as loaded), the cell-status counts (`n_cells_requested`, `n_cells_ok`, `n_cells_failed`, `n_cells_not_implemented`), `wall_clock_seconds` and `timestamp_utc`, and the **factorlasso**, Python, **NumPy**, and **pandas** versions with the platform string. The cell-status counts and version stamps let a reviewer verify whether a re-run reproduced the original study and reconcile any differences against the reference run of Section 5.

**Affiliation:**

Artur Sepp
Global Head of Quantitative Analytics
LGT Bank
E-mail: artur.sepp@lgt.com
URL: https://github.com/ArturSepp/factorlasso

Mika A. Kastenholz
Global Head of Investment Solutions
LGT Bank
E-mail: mika.kastenholz@lgt.com

*Journal of Statistical Software* https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd